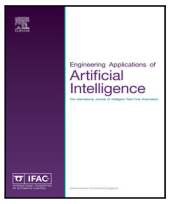
Engineering Applications of Artificial Intelligence 158 (2025) 111283



Contents lists available at ScienceDirect

Engineering Applications of Artificial Intelligence

journal homepage: www.elsevier.com/locate/engappai



Research paper



Global–local adaptive resampling strategy for enhancing the performance of Physics-informed neural networks

Lei Gao [a], Dan Zhang [a,b,*], Yaoran Chen [b,d], Xiaowei Li [a], Chunxin Li [a], Yan Peng [c,e]

[a] School of Mechatronic Engineering and Automation, Shanghai University, Shanghai, 200444, China
[b] Institute of Artificial Intelligence, Collaborative Innovation Center for the Marine Artificial Intelligence, Shanghai University, Shanghai, 200444, China
[c] Shanghai Artificial Intelligence Laboratory, Shanghai, 200444, China
[d] School of Future Technology, Shanghai University, Shanghai, 200444, China
[e] Institute of Artificial Intelligence, Shanghai University, Shanghai, 200444, China

ARTICLE INFO

Keywords:
Artificial intelligence
Physics-informed neural networks
Partial differential equations
Monte Carlo integration
Adaptive resampling strategy

ABSTRACT

In recent years, the rapid development of artificial intelligence (AI) has brought innovative solutions to the field of engineering. Physical-informed neural networks (PINNs) have provided a new paradigm for solving partial differential equations (PDEs). However, PINNs are extremely sensitive to the number and distribution of collocation points. Insufficient collocation points or uneven distribution may lead can lead to solution failure. To address this issue, this study proposes a global–local adaptive resampling strategy (GLAR) that combines Monte Carlo integration with PINNs. During the training process, the collocation points are resampled in both global and local regions according to the Monte Carlo integral values to improve the accuracy of the model. Numerical experiments show that GLAR-PINNs is comparable to existing resampling methods when dealing with linear PDEs (such as the Diffusion and Wave equations). When it comes to solving nonlinear PDEs (such as the Burgers, Korteweg–de Vries, and Allen–Cahn equations), the accuracy is improved has been improved by 14.65 times, 38.04 times, and 7.14 times, respectively. In addition, we applied this strategy to reconstruct the flow field around a two-dimensional triangular cylinder, and the relative error of the reconstructed velocity field was less than 0.001. This significantly promoting significantly promotes the application and development of artificial intelligence in the engineering field.

1. Introduction

Since the concept of Physics-informed neural networks (PINNs) was introduced by Raissi and Karniadakis (2018), Raissi et al. (2017a,b,c), Maziar Raissi et al. (2017) and Raissi et al. (2018, 2019), its applications in scientific computing and engineering have witnessed rapid expansion. The applications span a broad spectrum of fields, including fluid dynamics (Eivazi et al., 2022; Joshi et al., 2023), materials science (Yin et al., 2021; Li et al., 2024; Rouhollahi et al., 2024), biomedicine (Kumar et al., 2023; Francois and Laubscher, 2023), solid mechanics (Wang et al., 2024; Cao and Wang, 2025), optics (Chen et al., 2020), and electromagnetics (Baldan et al., 2023). To address the challenges in diverse scientific computing and application scenarios and to further enhance the accuracy and efficiency of PINNs, researchers have proposed several improvement strategies. These include the development of Generative Pre-trained Physics-informed neural networks (GPT-PINNs) (Chen and Koohy, 2024), Gradient-enhanced Physics-informed neural networks (G-PINNs) (Yu et al., 2022), Backward-compatible Physics-informed neural networks (BC-PINN) (Mattey and Ghosh, 2022; Lin and Chen, 2024), and NSFnets-PINNs (Jin et al., 2020), which is specifically designed to solve the Navier–Stokes equations.

In recent years, the extensive application of PINNs in numerical computation and engineering has highlighted the substantial potential of artificial intelligence (AI) in addressing practical problems. This is particularly evident in the reconstruction of flow field information, where the challenges of obtaining experimental data and the high computational cost and low efficiency of traditional numerical methods have been significant hurdles. PINNs have demonstrated a notable advantage by reconstructing high-precision flow fields using sparse data combined with physical information. For example, to address the issue of reconstructing flow fields from low-fidelity data under high Reynolds number conditions, Fan et al. (2023) skillfully introduced reduced-order differential equations for viscous fluid motion and low-fidelity Reynolds-Averaged Navier–Stokes equations as constraints in the loss function. This approach successfully enabled high-precision

* Corresponding author at: School of Mechatronic Engineering and Automation, Shanghai University, Shanghai, 200444, China.
E-mail address: dan.zhang@shu.edu.cn (D. Zhang).

https://doi.org/10.1016/j.engappai.2025.111283
Received 26 December 2024; Received in revised form 6 March 2025; Accepted 22 May 2025
Available online 10 June 2025
0952-1976/© 2025 Elsevier Ltd. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

super-resolution reconstruction of the flow field. Yao et al. (2024) effectively utilized PINNs to reconstruct sparse storm fields with limited observational data, achieving satisfactory predictive outcomes. Additionally, Hou et al. (2024) proposed an adaptive weighting method to balance the various components of the loss function in PINNs, thereby enhancing the accuracy of flow field reconstruction. This method has been successfully applied to the reconstruction of ship propeller wake fields.

However, when faced with more complex engineering and scientific problems, PINNs may fall into local optima or fail to converge, which affects the accuracy and efficiency of the solution. To address these issues, researchers have investigated PINNs from various perspectives. For example, Xiao et al. (2024) pointed out that the computational cost of automatic differentiation is too high and proposed using least-squares finite difference instead of automatic differentiation to compute the required differential operators in PINNs. This method relies solely on the network's output to calculate derivatives, thereby avoiding a series of matrix operations and improving the precision and efficiency of PINNs. Additionally, Huang et al. (2024) also noted the high computational and storage costs of automatic differentiation and employed the Chebyshev spectral method as a replacement, significantly enhancing computational efficiency. From the perspective of collocation points, Yang et al. (2024) proposed an adaptive collocation point sampling strategy, which made the distribution of collocation points more accurate and controllable, thereby improving the solving efficiency and accuracy of PINNs.

Although PINNs show strong performance in solving PDEs, they often need to be retrained when faced with PDEs with different parameters, which undoubtedly limits their application scope. To address this challenge, researchers have made improvements to PINNs from various perspectives. Chen and Koohy (2024) proposed a method of generating pre-trained PINNs to solve the problem of parameterized PDEs. Cho et al. (2024) proposed a parametric PINNs by explicitly encoding PDEs with different parameters, thereby further realizing the parametric modeling of PDEs. Xiang et al. (2025) proposed the Physics-informed Neural Implicit Flow framework, which incorporates Neural Implicit Flow into a physics-informed learning framework to handle the reduced-rank representation of parametric spatio-temporal data in a grid-independent manner. This framework enables efficient handling of variable coefficients in PDEs and effectively captures the intricate dynamics of parametric PDEs.

Although these improved PINNs perform well in their respective fields and application scenarios, their sampling strategy for collocation points is usually random uniform sampling. This strategy may achieve high accuracy in solving simple partial differential equations (PDEs). However, when dealing with complex nonlinear PDEs involving higher-order derivatives, it may encounter situations where the solution fails to converge or even diverges (Guo et al., 2023; W and Zhao, 2021). Therefore, the sampling and distribution of collocation points are very important, which has been theoretically demonstrated in some literature (Tang et al., 2023; Gao et al., 2023a; Mao et al., 2020). For example, Subramanian et al. (2023) investigated the effect of the position of collocation points on the performance of PINNs. They discovered that adjusting the collocation points to appropriate positions can remarkably enhance the performance of PINNs. Gao et al. (2023b) pointed out that fixed random uniform sampling collocation points may not effectively capture the solutions of PDEs. Therefore, they proposed an adaptive collocation points sampling strategy for failure formed PINNs (f-PINNs), which can significantly improve the accuracy of numerical solutions.

The significance of collocation points has gradually drawn the attention of researchers. For example, the team led by Lu et al. (2021) first added more collocation points in regions with larger residuals of PDEs and proposed a sampling strategy based on residual adaptive refinement (RAR). This strategy enhanced the accuracy of the PINNs solution. Subsequently, team members Wu et al. (2023) proposed collocation points sampling strategies based on residual adaptive distribution (RAD) and residual adaptive refinement and distribution (RAR-D) on the basis of RAR. RAD and RAR-D can dynamically optimize the distribution of collocation points during the training process according to the characteristics of PDEs, thereby improving the accuracy of the PINNs solution. Moreover, Liu et al. (2024) pointed out that the RAR strategy only focuses on collocation points within the solution domain, while neglecting the attention to sampling points at the boundary. Therefore, they proposed an expected improved RAR (EI-RAR) strategy. This strategy increases the attention to sampling points at the boundary of the solution domain and integrates the attention mechanism with the sampling-point generation algorithm. It strengthens the connection between newly added sampling points and existing sampling points, improving the accuracy of PINNs in solving PDEs with sharp solutions. In addition, Nabian et al. (2021) sampled the collocation points based on their proportional distribution to the loss function, aiming to improve the training efficiency and convergence performance of PINNs.

In the past two years, to effectively enhance the accuracy, efficiency, and robustness of Physics-informed Neural Networks (PINNs), researchers have made various degrees of improvements and optimizations to the sampling and distribution strategies of collocation points within the solution domain. For example, Visser et al. (2024) proposed a point adaptive collocation method for artificial neural networks (PACMANN) to tackle the problem of poor scaling ability of adaptive residual collocation point sampling methods in high-dimensional problems. Numerical experiments demonstrate that this method outperforms existing methods when solving high-dimensional problems. Zhang et al. (2024) put forward an annealing adaptive importance sampling (AAIS) strategy to address the issue of local-minimum singularity resulting from insufficient distribution of collocation points in certain regions. This strategy approximates the discrete PDEs residuals by strategically sampling additional points in areas with high residuals, thereby enhancing the effectiveness and accuracy of PINNs. Jiao et al. (2024) generated a Gaussian mixture distribution using the current residual information and employed it to add additional collocation points for sampling. This method can accelerate the convergence rate of the PINNs loss function and improve its solution accuracy.

Although the distribution of collocation points and sampling strategies mentioned above have enhanced the accuracy, efficiency, and convergence rate of PINNs in solving complex PDEs, endowing PINNs with certain robustness and generalization capabilities, these sampling strategies still present two main issues: (1) They typically concentrate solely on local regions with large residuals, while overlooking the distribution and quantity of collocation points in the global (the entire solution domain). (2) These methods necessitate an appropriate number of initial collocation points. Notably, when the number of initial collocation points is extremely small, it may cause the method to fail to converge or even be unable to obtain a solution.

Therefore, this study proposes a global–local adaptive resampling (GLAR) strategy to address the aforementioned problems associated with an extremely small number of initial collocation points. This strategy not only focuses on the distribution of collocation points in local regions but also takes into account the distribution of global collocation points. By using Monte Carlo integration as the benchmark for evaluating the accuracy of PINNs models, the course of the next round training is determined based on Monte Carlo integration values during the training process. If the training is to continue, a certain number of collocation points are first randomly added across the entire computational domain. Subsequently, more collocation points are distributed in regions with larger residuals of PDEs. The training is terminated when the Monte Carlo integration value reaches the predefined threshold.

The GLAR strategy proposed in this study successfully solves several different types of PDEs, including the linear Diffusion equation,

Wave equation, and the nonlinear Burgers equation, Korteweg–de Vries equation, and Allen–Cahn equation. Moreover, it is also applied to the reconstruction of the flow field around a two-dimensional triangular cylinder. To highlight the outstanding performance of the GLAR strategy, ablation contrast experiments were conducted in each numerical experiment. These experiments involved two approaches: randomly and uniformly adding collocation points across the entire computational domain (referred to as the global adaptive resampling strategy, GAR in this paper) and adding collocation points based on the residual adaptive refinement sampling strategy (referred to as the local adaptive resampling strategy, LAR in this paper). Through the numerical experiments on these equations, it is evident that the GLAR strategy can effectively enhance the accuracy of PINNs. Additionally, it endows PINNs with certain generalization and robustness capabilities.

The rest of this paper is organized as follows: Section 2 briefly introduces PINNs, GLAR and related methods. In Section 3, numerical experiments were carried out by applying the GLAR strategy on different types of PDEs and the GAR and LAR for ablation comparison to verify the effectiveness and generalization ability of the GLAR strategy. Section 4 summarizes the performance of the GLAR strategy and puts forward the main conclusions.

2. Methods

2.1. Physics-informed neural networks

PINNs incorporate physical laws, experimental data, and prior information into the neural network architecture. They approach the solution of the function $u(x,t)$ through a continuous iterative training process. The solution obtained by this method is not only continuous but also smooth. To illustrate this principle briefly, we take a PDE as an example.

$$u_t + \mathcal{N}[u] = 0, x \in \Omega, t \in [0,T], \tag{1}$$

$$u(x,0) = I(x), x \in \Omega, t = 0, \tag{2}$$

$$u(x,t) = B(x), x \in \partial\Omega, t \in [0,T] \tag{3}$$

where, the variables x and t represent the spatial location and time point, respectively. u_t refers to the derivative of function u with respect to time t, and $\mathcal{N}[\cdot]$ represents a differential operator. $I(x)$ and $B(x)$ represent initial conditions and boundary conditions, respectively. The core goal of PINNs is to calculate the solution $u(x,t)$ of PDEs under given initial and boundary conditions.

In the PINNs framework, the neural network $f(x,t;\theta)$ is repeatedly trained to obtain an approximate solution that is consistent with the label data and the laws of physics. Here, we consider a neural network with L layers. Its parameter set is represented as θ, and the network contains $L-1$ hidden layers, along with an input layer and an output layer.

$$x^0 = [x,t]^T, \tag{4}$$

$$H_l(x^{l-1}) = w^l x^{l-1} + b^l, l = 1,2,\dots,L, \tag{5}$$

$$f(x,t;\theta) = (H_L \circ \dots \circ \sigma \circ H_1)x^0 \tag{6}$$

where x^0 is the input vector, $\theta = \{w^l, b^l\}_{l=1}^{L}$ is a set containing weights and biases of all layers of the neural network. $\circ$ is the combination operator, σ is the hyperbolic tangent activation function $tanh(x)$.

Then, we substitute the solution $f(x,t;\theta)$ predicted by the neural network in each iteration into Eq. (1) to construct the residual function $R(x,t;\theta)$.

$$R(x,t;\theta) = f_t(x,t;\theta) + \mathcal{N}[f(x,t;\theta)] \tag{7}$$

where, the neural network $f_t(x,t;\theta)$ represents the partial derivative with respect to the variable t, which can be computed using automatic differentiation techniques. The loss function typically consists of the following four main components.

1. The loss function L_D driven by labeled data.

$$L_D = \frac{1}{N_D}\sum_{i=1}^{N_D} |u_D^i - f_D(x^i,t^i;\theta)|^2 \tag{8}$$

2. The loss function L_I driven by the initial conditions.

$$L_I = \frac{1}{N_I}\sum_{i=1}^{N_I} |u_I^i - f_I(x^i,t^i;\theta)|^2 \tag{9}$$

3. The loss function L_B driven by the boundary conditions.

$$L_B = \frac{1}{N_B}\sum_{i=1}^{N_B} |u_B^i - f_B(x^i,t^i;\theta)|^2 \tag{10}$$

4. The loss function L_R driven by physical constraints (residual function).

$$L_R = \frac{1}{N_R}\sum_{i=1}^{N_R} |R(x^i,t^i;\theta)|^2 \tag{11}$$

Integrating these loss functions guarantees that the predictions of PINNs not only align with the labeled data but also adhere to the physical laws. The integrated loss function typically takes on the following two forms.

1. Data-driven (including labeled data).

$$L(\theta) = w_D L_D(\theta) + w_I L_I(\theta) + w_B L_B(\theta) + w_R L_R(\theta) \tag{12}$$

2. Data-free driven.

$$L(\theta) = w_I L_I(\theta) + w_B L_B(\theta) + w_R L_R(\theta) \tag{13}$$

where w_D, w_I, w_B, w_R represent the weight coefficients corresponding to the data-driven loss, initial condition loss, boundary condition loss, and residual loss, respectively. These weights may play an important role in solving particular PDEs. In order to highlight the effectiveness of the method proposed in this paper, we will not discuss the specific values of these weights in depth, but set them to 1. Additionally, N_D, N_I, N_B, N_R indicates the number of data points involved in each loss term.

In the training process of PINNs, we usually use optimization algorithms such as Adam, SGD or L-BFGS to update the parameter θ of neural network. By minimizing the loss function $L(\theta)$, we try to make its value as close to zero as possible. Once this goal is achieved, we can consider that the neural network $f(x,t;\theta)$ has become the approximate solution $u(x,t)$ of Eq. (1).

2.2. Monte Carlo integration

Monte Carlo method is a computing technique based on random sampling, which is widely used in mathematics, physics and engineering fields. It shows unique advantages in solving complex integral, optimization problems and system simulation, such as using Monte Carlo method to approximate complex fractional partial differential equations (fPDEs) (Leonenko and Podlubny, 2022a,b). In the past two years, with the rapid development of PINNs, scholars have combined Monte Carlo method with PINNs to solve PDEs (Guo et al., 2022; Wang and Karniadakis, 2024), and made some achievements in accuracy and efficiency. It can be seen that Monte Carlo method can promote the development and application of PINNs to a certain extent.

In this paper, the Monte Carlo method is used to integrate the residual $R(x,t;\theta)$. This method provides a quantitative measure of the overall distribution of residuals through random sampling, allowing for a comprehensive evaluation of the performance of the PINNs model. To quantitatively evaluate the performance of PINNs during training, we

introduced a threshold ε. When the average Monte Carlo integral of all collocation points is lower than the set threshold ε, the performance of PINNs is considered to be satisfactory.

$$F(x,t) = \frac{1}{N_M}\sum_{i=1}^{N_M}\frac{|R(x^i,t^i;\theta)|}{pdR(x^i,t^i;\theta)} \le \varepsilon, \tag{14}$$

$$pdR(x,t;\theta) = \frac{1}{\Omega} \tag{15}$$

where N_M is the number of collocation points in calculating Monte Carlo integration, $pdR(x,t;\theta)$ is the probability density, and Ω is the space.

To ensure the accuracy of Monte Carlo integration, according to Bernoulli's theorem of large numbers, we need to generate a large number of collocation points $\{x^i,t^i\}_{i=1}^{N_M}$ in the computational domain of PINNs. The purpose of doing this is to ensure that the average value of the residual function $R(x,t;\theta)$ in the computational domain can be infinitely close to zero, thereby meeting the accuracy requirements of PINNs in training.

$$\lim_{N_M\to\infty} F(x,t) = R(x,t;\theta) = 0 \tag{16}$$

Through Monte Carlo method, we can effectively evaluate the model accuracy of PINNs in the training process. This method can not only help us improve the prediction ability of the model, but also ensure the physical consistency of the model in the whole training process.

2.3. Global–local adaptive resampling strategy

Aiming at the existing problem of adaptive resampling of residual collocation points, we propose a global–local adaptive resampling algorithm (GLAR) to improve the accuracy of PINNs. In the training process, we use the mean value of the Monte Carlo integral as the evaluation benchmark for model accuracy and determine whether to continue training by comparing it with the preset threshold ε. This method not only optimizes the distribution of residual points, but also significantly improves the accuracy of PINNs.

Algorithm 1: GLAR-PINNs

1: Initial collocation points set T_{RC} obtained by Latin hypercube sampling method;
2: Generate initial and boundary condition datasets;
3: Set the threshold ε and the maximum collocation points number S;
4: Generate collocation points set T_M for Monte Carlo integration calculation, where $N_{T_M} \to \infty$;
5: Training PINNs;
6: Calculate Monte Carlo integration $F(x,t)$.
7: While $F(x,t) \ge \varepsilon$ Do
8: Randomly sample m collocation points in the entire computational domain as S^1_{RC};
9: Substitute the collocation points T_M into the residual function $R(x,t;\theta)$and calculate the residual values;
10: Sort the calculated residual values and elect n collocation points with larger residual values as S^2_{RC}, where $n \ll N_{T_M}$;
11: Merge S^1_{RC} and S^2_{RC} with T_{RC}, i.e. $S^1_{RC} \cup S^2_{RC} \cup T_{RC} = T_{RC}$;
12: if $N_{T_{RC}} \ge S$
13: Break
14: end if
15: Training PINNs;
16: Calculate Monte Carlo integration $F(x,t)$.
17: end while
18: Return the neural network solution $u(x,t;\theta)$

GLAR-PINNs first undergo initial training. After training, the accuracy of the model is evaluated using Monte Carlo integration. If the conditions are not met, we randomly sample m collocation points in the entire computational domain and add them to the initial collocation point set T_{RC}. Next, we substitute the set of collocation points T_M into the residual function $R(x,t;\theta)$ to calculate the residual values of each collocation point. We then select n points with larger residual values and add them to the initial collocation point set T_{RC}. Given the large number of collocation points in T_M and the fact that n is much smaller than the number of points in T_M, according to Bernoulli's law of large numbers, the selected n collocation points are primarily concentrated in one or several specific regions. Therefore, our algorithm can identify the local regions with larger residual values within each training cycle and allocate more collocation points within these regions to enhance the model's attention to areas with larger residual values. This training process continues until the stopping condition is met, at which point the training ends. The detailed process steps are shown in Algorithm (1). Through this strategy, we not only reduce the dependence of existing resampling strategies on the initial number of collocation points but also improve the generalization ability and solution accuracy of PINNs.

In order to verify the effectiveness of the GLAR method and highlight its excellent performance, the ablation experiments of GAR and LAR were carried out on different types of PDEs. The algorithmic framework of these two methods is similar to Algorithm (1), so it will not be repeated here.

2.4. Performance evaluation benchmark

In order to verify the accuracy of the PINNs model trained by the algorithm proposed in this study, the absolute error (17) is used to quantify the difference between the predicted value and the exact solution. In addition, the overall performance of the model is evaluated by L_2 relative error (19). In Section 3.6, we also use relative errors (18) to evaluate the accuracy of GLAR-PINNs reconstruction of flow field information. With the help of these three methods, we can comprehensively evaluate whether the model achieves the required accuracy level in solving complex physical problems.

$$Error_A = |u(x^i,t^i) - \hat{u}(x^i,t^i)|, \tag{17}$$

$$Error_R = |\frac{u(x^i,t^i) - \hat{u}(x^i,t^i)}{u(x^i,t^i)}|, \tag{18}$$

$$Error_{L_2} = \frac{\sqrt{\sum_{i=1}^{N}|u(x^i,t^i) - \hat{u}(x^i,t^i)|^2}}{\sqrt{\sum_{i=1}^{N}|u(x^i,t^i)|^2}}, \tag{19}$$

where $u(x^i,t^i)$ is the exact value, $\hat{u}(x^i,t^i)$ is the predicted value of the model, and N is the number of test data points.

3. Numerical experiment

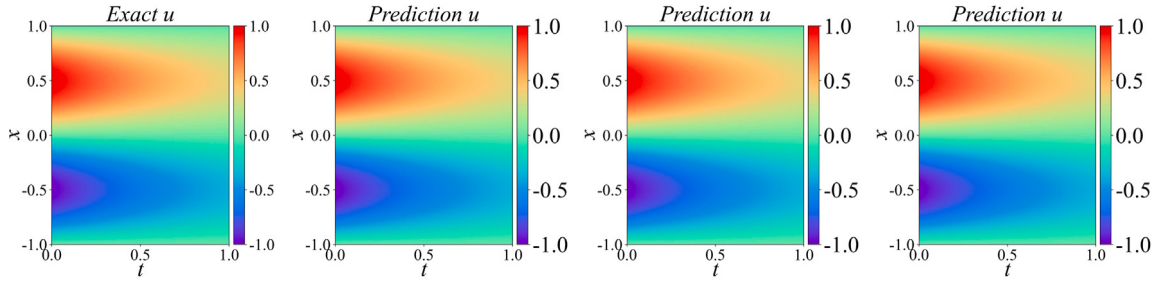
In order to verify the effectiveness of the proposed GLAR method, numerical experiments were carried out for six different types of equations, including linear Diffusion equation, Wave equation and nonlinear Burgers equation, Korteweg–de Vries equation, Allen–Cahn equation, as well as the reconstruction of flow field around a two-dimensional triangular cylinder. The exact solutions of linear Diffusion equation and Wave equation can be obtained by analytical formula, while the exact solutions of Burgers equation, Korteweg–de Vries equation and Allen–Cahn equation are obtained by numerical solution using Chebfun tool in MATLAB. The exact solution of the flow around a two-dimensional triangular cylinder is obtained by the NekTar solver. The neural network parameters of all numerical experiments in this paper are shown in Table 1. The experimental code is developed based on pytoch framework and runs on NVIDIA geforce RTX 4090 GPU.

3.1. Diffusion equation

The one-dimensional Diffusion equation possesses linear characteristics, and its solutions typically display good smoothness. Given the high efficiency of PINNs in solving such equations, selecting this equation as the object of numerical experiments can rapidly verify the effectiveness of our proposed method. For further evaluation, we

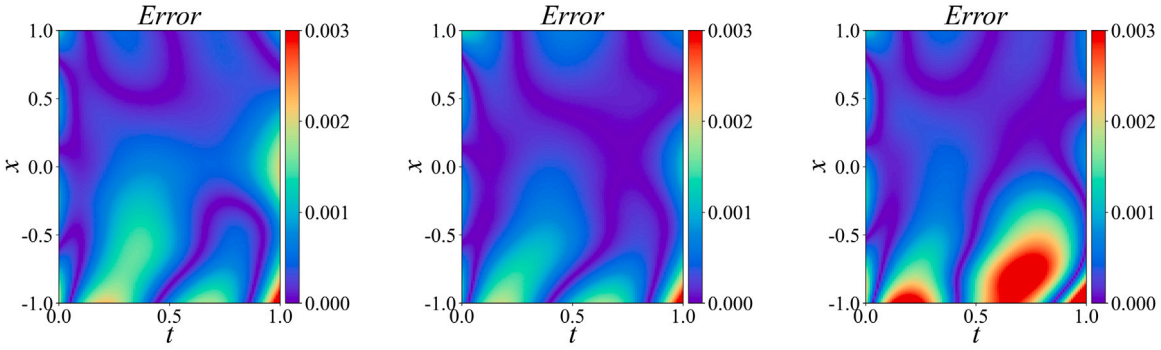
Table 1
Parameters of neural network training.

Section number and problem	Neural network structure	Optimizer	Adam	
			Iterations	Learning rate
3.1 Diffusion equation	[2] + 4 ∗ [20] + [1]	Adam+L-BFGS	1000	0.001
3.2 Wave equation	[3] + 4 ∗ [50] + [1]	Adam+L-BFGS	3000	0.001
3.3 Kovasznay Flow equation	[2] + 6 ∗ [20] + [1]	Adam+L-BFGS	1000	0.001
3.4 Korteweg–de Vries equation	[2] + 6 ∗ [128] + [1]	Adam+L-BFGS	3000	0.001
3.5 Allen–Cahn equation	[2] + 9 ∗ [64] + [1]	Adam+L-BFGS	2000	0.001
3.6 Navier–Stokes equation	[3] + 4 ∗ [50] + [1]	Adam+L-BFGS	2000	0.001



(a) Exact solution (b) GLAR-PINNs (c) GAR-PINNs (d) LAR-PINNs

Fig. 1. Two-dimensional cloud map of the Diffusion equation using three methods. (a) Exact solution; (b) Predictive solution using GLAR-PINNs; (c) Predictive solution using GAR-PINNs; (d) Predictive solution using LAR-PINNs.



(a) GLAR-PINNs (b) GAR-PINNs (c) LAR-PINNs

Fig. 2. Absolute error cloud maps for solving the Diffusion equation using three methods. (a) GLAR-PINNs; (b) GAR-PINNs; (c) LAR-PINNs.

Table 2
Numerical experimental results of Diffusion equation.

Methods		GLAR-PINNs	GAR-PINNs	LAR-PINNs
Initial collocation points		10	10	10
Final collocation points		70	40	70
Times		25 s	27 s	20 s
Monte Carlo integration		0.0088	0.0068	0.0148
Absolute error	Mean	0.0005	0.0003	0.0006
	Std	0.0004	0.0003	0.0007
L_2 relative error		0.0015	0.0011	0.0022

investigated a one-dimensional linear Diffusion equation with a source term, which has the following specific form.

$$u_t - u_{xx} = e^{-t}(-sin(\pi x) + \pi^2 sin(\pi x)), x \in [-1, 1], t \in [0, 1], \tag{20}$$

$$u(x, 0) = sin(\pi x), \tag{21}$$

$$u(-1, t) = u(1, t) = 0 \tag{22}$$

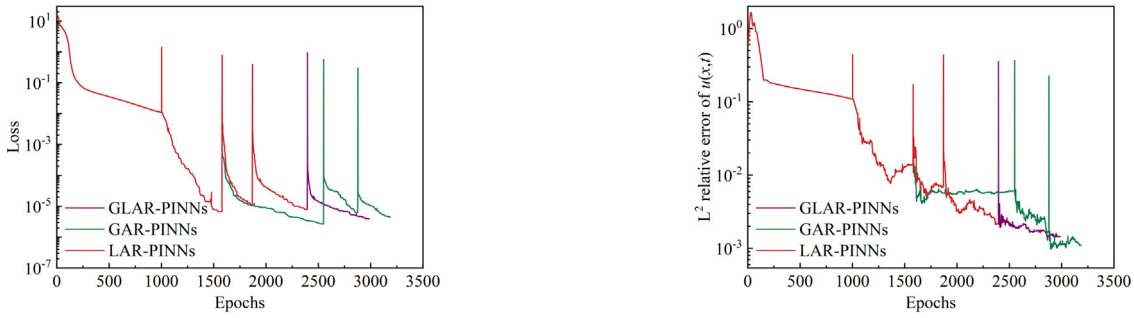
The analytical solution of this equation is

$$u(x, t) = sin(\pi x)e^{-t} \tag{23}$$

In this numerical experiment, the threshold for Monte Carlo integration is set to $\varepsilon = 0.01$, and the maximum number of collocation points is limited to $S = 1000$. The training dataset contains $N_I = 25$ initial points, $N_B = 100$ boundary points, and 10 initial collocation points. During each resampling, add 10 collocation points to both the global and local regions. In addition, there are $N_M = 50000$ collocation points used for calculating Monte Carlo integration.

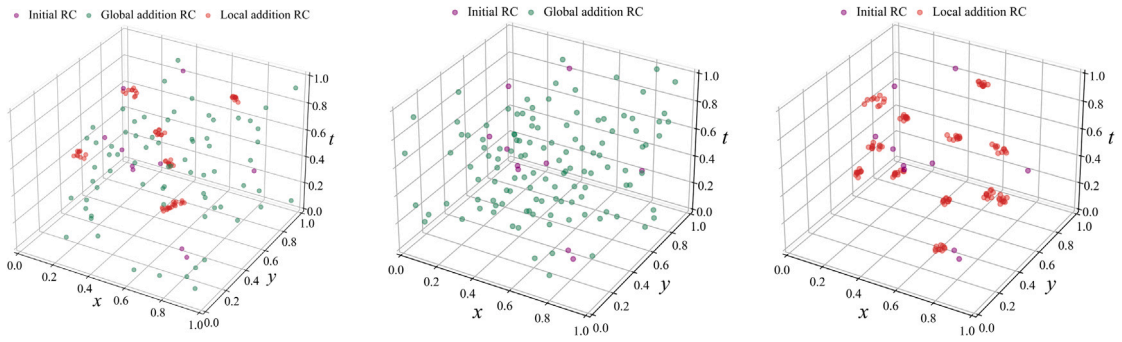
When GLAR-PINNs training stops, the total number of collocation points reaches 70. On this basis, keeping the Monte Carlo integration threshold unchanged, the maximum number of collocation points was set to 70, and GAR-PINNs and LAR-PINNs were trained separately. The numerical experimental results of the three methods are detailed in Table 2. From the table, it can be seen that although the GAR-PINNs model has relatively high solution accuracy, the L_2 relative errors and mean absolute error of the three methods are very close, indicating that their solution results are similar, as shown in Fig. 1.

Fig. 2 shows the absolute error cloud maps of three methods for solving the diffusion equation. Observation shows that the error regions of GLAR-PINNs and GAR-PINNs methods are similar, with no significant difference. However, the LAR-PINNs method has a wider area of regions with larger absolute error, indicating that its solution performance is relatively poor. Fig. 3 depicts the changing trends of loss function and L_2 relative error during the training process for three



(a) Loss function (b) L_2 relative error

Fig. 3. Evolution of loss function and L_2 relative error during solving Diffusion equation by three methods. (a) Loss function; (b) L_2 relative error.



(a) GLAR-PINNs (b) GAR-PINNs (c) LAR-PINNs

Fig. 4. Final distribution of collocation points for solving the Wave equation using three methods. (a) GLAR-PINNs; (b) GAR-PINNs; (c) LAR-PINNs. RC denotes the collocation points, where purple indicates the initial RC, green indicates the global addition RC, and red indicates the local addition RC.

methods. From Fig. 3(a), it can be seen that the loss function values of the three methods can all be reduced to the order of 10^{-5}, demonstrating similar optimization performance. However, from Fig. 3(b), it can be seen that the L_2 relative error of LAR-PINNs is significantly higher than that of GLAR-PINNs and GAR-PINNs, and the GAR-PINNs method performs the best.

From this numerical experiment, it can be seen that although the GAR-PINNs method performs the best in solving one-dimensional Diffusion equations, the difference in solving accuracy among the three methods is only on the order of 10^{-3}, which is almost negligible. Therefore, it can be considered that the three methods have similar solving performance. This also indirectly reflects the effectiveness of the GLAR-PINNs method.

3.2. Wave equation

In this section, we use the GLAR-PINNs method to solve two-dimensional Wave equations to verify its effectiveness and accuracy in solving linear PDEs. The general form of the two-dimensional Wave equation we consider is as follows.

$$u_{tt} = c^2(u_{xx} + u_{yy}), x \in [0,1], t \in [0,1], \tag{24}$$

$$u(x,y,0) = sin(\pi x)sin(\pi y), \tag{25}$$

$$u_t(x,y,0) = 0, \tag{26}$$

$$u(0,y,t) = u(1,y,t) = u(x,0,t) = u(x,1,t) = 0 \tag{27}$$

The analytical solution of this equation is

$$u(x,t) = cos(\sqrt{2}\pi t)sin(\pi x)sin(\pi y) \tag{28}$$

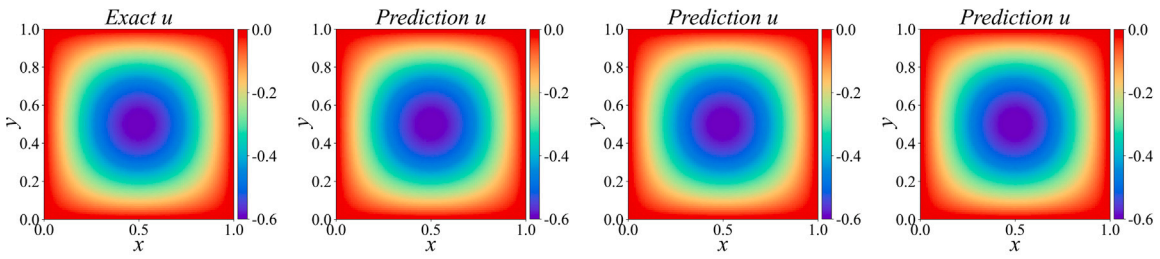
Table 3
Numerical experimental results of Wave equation.

Methods		GLAR-PINNs	GAR-PINNs	LAR-PINNs
Initial collocation points		10	10	10
Final collocation points		150	130	150
Times		418 s	334 s	393 s
Monte Carlo integration		0.0075	0.0082	0.0187
Absolute error	Mean	0.0011	0.0010	0.0015
	Std	0.0006	0.0007	0.0012
L_2 relative error		0.0050	0.0048	0.0055

In this numerical experiment, the wave velocity c is set to 1. The training dataset consists of initial conditions $N_I = 100$, boundary conditions $N_B = 200$, and 10 initial collocation points. When the Monte Carlo integral value of the model drops to the preset threshold, the cumulative number of collocation points reaches 150. We set 150 as the maximum threshold for the number of collocation points and conducted ablation comparison experiments between GAR-PINNs and LAR-PINNs based on this.

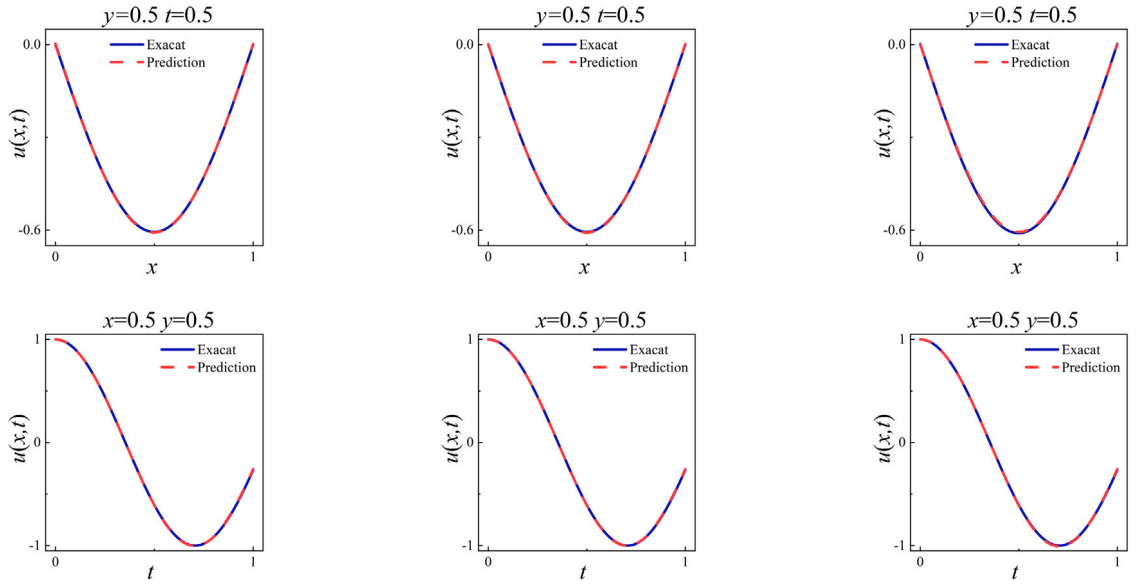
Fig. 4 shows the final distribution of collocation points at the end of training for three methods. We noticed that a portion of the collocation points in GLAR-PINNs are uniformly distributed within the computational domain, while another portion is concentrated in specific regions. In contrast, the collocation points of GAR-PINNs are randomly distributed within the computational domain, while the collocation points of LAR-PINNs are mainly concentrated in local regions. The solution results of the three methods are shown in Figs. 5 and 6, respectively. From these graphs, it can be seen that there is not much difference in the solution results of the three methods, which to some extent verifies the effectiveness of the GLAR-PINNs method in solving linear PDEs.

The absolute error cloud map in Fig. 7 reveals that LAR-PINNs have larger errors in multiple regions, whereas GLAR-PINNs exhibit



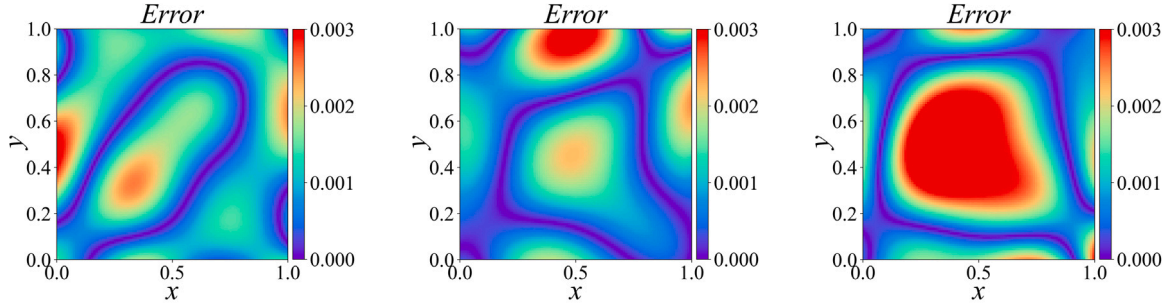
(a) Exact (b) GLAR-PINNs (c) GAR-PINNs (d) LAR-PINNs

Fig. 5. Two-dimensional cloud map of the wave equation at $t = 0.5$ s solved using three methods. (a) Exact solution; (b) GLAR-PINNs; (c) GAR-PINNs; (d) LAR-PINNs.



(a) GLAR-PINNs (b) GAR-PINNs (c) LAR-PINNs

Fig. 6. Curves of the Wave equation under different conditions solved using three methods. (a) GLAR-PINNs; (b) GAR-PINNs; (c) LAR-PINNs.



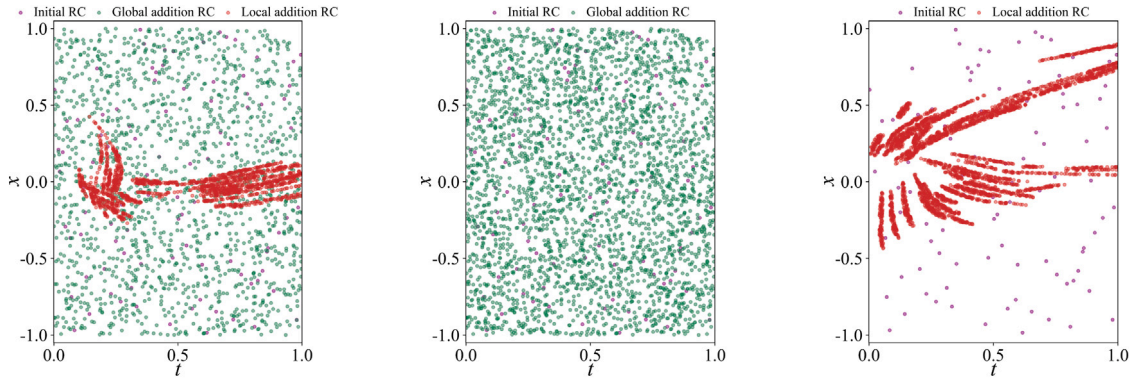
(a) GLAR-PINNs (b) GAR-PINNs (c) LAR-PINNs

Fig. 7. Absolute error cloud map of the Wave equation solved using three methods at $t = 0.5$ s. (a) GLAR-PINNs; (b) GAR-PINNs; (c) LAR-PINNs.

smaller errors. This disparity is mainly attributed to the limitations of LAR-PINNs in the distribution of collocation points. Specifically, the collocation points of LAR-PINNs are overly concentrated in specific regions while neglecting other regions, leading to an increase in errors in those ignored regions. Due to the relatively simple nature of the two-dimensional Wave equation, PINNs can accurately capture its physical characteristics. As a result, the absolute errors of the three methods are on the order of 10^{-3}. Additionally, the L_2 relative error and mean absolute error presented in Table 3 further confirm that the difference in accuracy among these three methods is confined to the order of 10^{-3}. It is noteworthy that the training times of these three methods do not differ significantly, ranging only from 1 to 2 min. The numerical experimental results indicate that the three methods demonstrate comparable performance in solving linear PDEs. Moreover, these results further validate the effectiveness and reliability of GLAR-PINNs in solving linear PDEs.

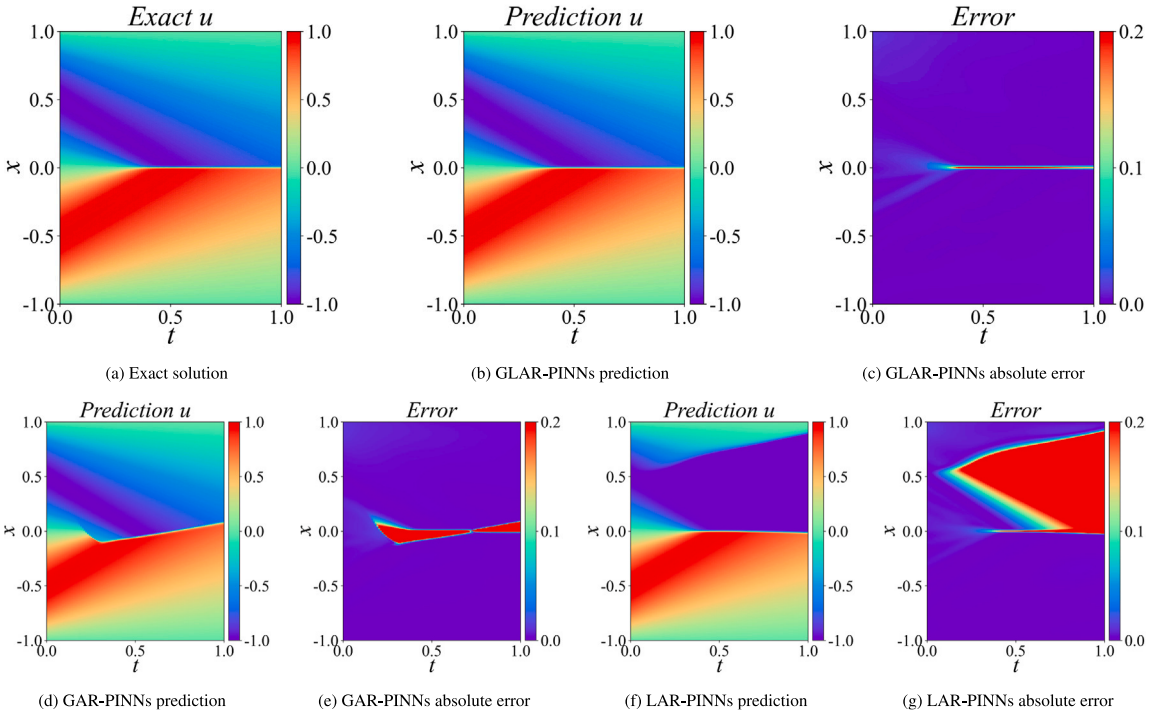
3.3. Burgers equation

The Burgers equation is extensively applied in fields such as fluid dynamics, meteorology, elasticity, and traffic flow. It encompasses convection, diffusion, and nonlinear terms. Owing to its complex nature,



(a) GLAR-PINNs (b) GAR-PINNs (c) LAR-PINNs

Fig. 8. Final distribution of collocation points for solving the Bourgers equation using three methods. (a) GLAR-PINNs; (b) GAR-PINNs; (c) LAR-PINNs. RC denotes the collocation points, where purple indicates the initial RC, green indicates the global addition RC, and red indicates the local addition RC.



(a) Exact solution (b) GLAR-PINNs prediction (c) GLAR-PINNs absolute error

(d) GAR-PINNs prediction (e) GAR-PINNs absolute error (f) LAR-PINNs prediction (g) LAR-PINNs absolute error

Fig. 9. Predicted solution and absolute error cloud map of the Burgers equation solved using three methods. (a) Exact solution; (b) GLAR-PINNs prediction; (c) GLAR-PINNs absolute error; (d) GAR-PINNs prediction; (e) GAR-PINNs absolute error; (f) LAR-PINNs prediction; (g) LAR-PINNs absolute error.

the solving process of the Burgers equation is challenging, and it has also emerged as a hot topic in the research of PINNs for solving nonlinear PDEs. How to accurately capture the nonlinear characteristics of the Burgers equation has become the key to evaluating the performance of PINNs in solving such problems. Its general form is as follows.

$$u_t + uu_x = \nu u_{xx}, x \in [-1, 1], t \in [0, 1], \tag{29}$$

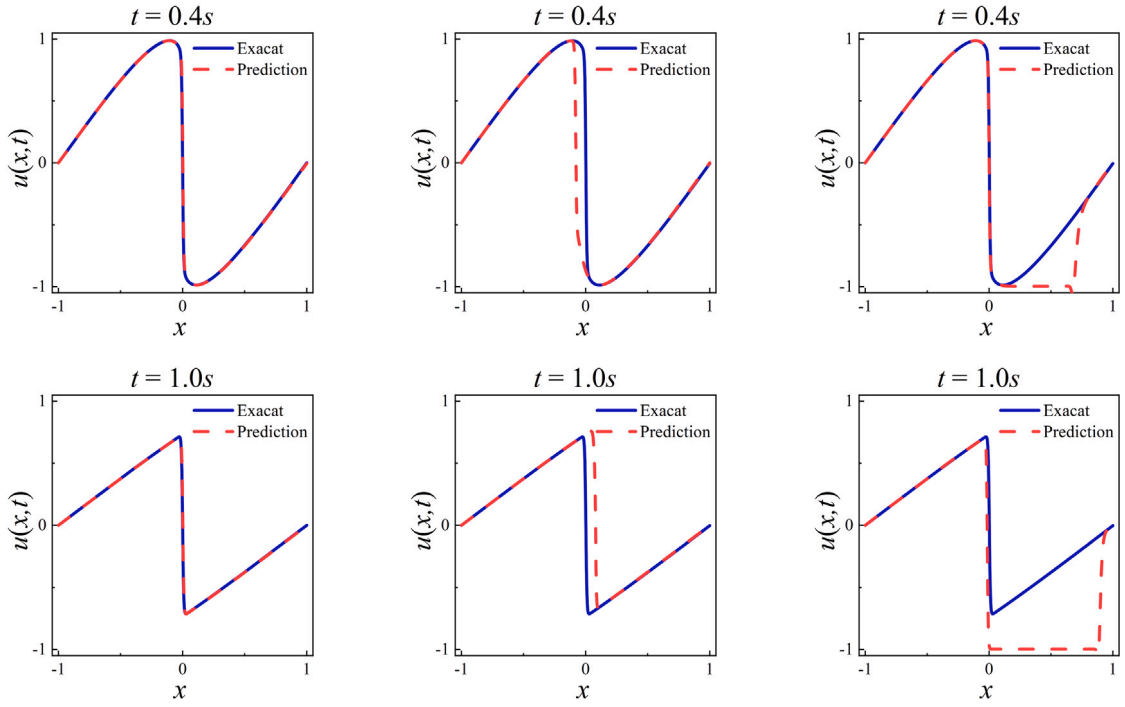
$$u(x, 0) = -sin(\pi x), \tag{30}$$

$$u(-1, t) = u(1, t) = 0 \tag{31}$$

In this paper, the parameter ν is set to $0.01/\pi$. The training data set contains 100 initial points, 1024 boundary points and 100 initial collocation points. In addition, the number of collocation points used to calculate Monte Carlo integration is 50 000. The Monte Carlo integration threshold of adaptive resampling is set to 0.01, and the total number of maximum collocation points is limited to 10 000. In each resampling process, we add 100 collocation points in the global and local regions until the stop condition is met.

We conducted the ablation experiments of GAR-PINNs and LAR-PINNs to ensure that other conditions remained unchanged, and set the threshold value of the maximum number of collocation points to 3000. The final distribution of collocation points of the three methods is shown in Fig. 8. It can be seen from the figure that the residual allocation points of GAR-PINNs are randomly distributed in the whole solution domain, while the residual allocation points of LAR-PINNs are mainly concentrated near $x = 0$, which is the location where the Burgers equation gradient changes greatly.

When the Monte Carlo integration value drops below 0.01, our GLAR-PINNs method adds a total of 3000 residual allocation points in the whole computational domain. Fig. 9(a), (b) shows the predictive solution of the Burgers equation solved by GLAR-PINNs and its corresponding absolute error cloud chart. From the error cloud chart, it can be seen that the error of this method is large near $x = 0$, and the maximum error is 0.2, while in other areas, the error is almost



(a) GLAR-PINNs (b) GAR-PINNs (c) LAR-PINNs

Fig. 10. Solutions of the Burgers equation at different times using three methods. (a) GLAR-PINNs; (b) GAR-PINNs; (c) LAR-PINNs.

zero. Fig. 10(a) shows the prediction results of GLAR-PINNs at different time points. The results show that the predicted solutions at all time points are in good agreement with the exact solutions. These results show that the proposed GLAR-PINNs method can solve the nonlinear Burgers equation with high accuracy, which verifies the effectiveness of the method.

The ablation results of GAR-PINNs and LAR-PINNs are shown in Fig. 9(d), (e), (f), (g). It is observed that when the residual allocation points of GAR-PINNs increase to 3000, the absolute error near $x = 0$ is large, while the accuracy of other positions is good, which is consistent with the characteristics of Burgers equation. The collocation points added by LAR-PINNs are mainly concentrated near $x = 0$ and the upper part of the solution domain (as shown in Fig. 8(c)), while the lower part has a large error value due to the less distribution of collocation points (as shown in Fig. 9(g)). The prediction results of the two methods at different time points are shown in Fig. 10(b), (c). It can be seen from the figure that the predicted solution and the exact solution are not completely consistent. The numerical experiments in this part show that under the condition of the same residual allocation points, the proposed GLAR-PINNs method can obtain higher accuracy prediction solutions through different residual allocation points distribution, which verifies the effectiveness of this method.

Next, we set the stopping conditions of GAR-PINNs and LAR-PINNs to be the same as those of GLAR-PINNs, and conduct numerical experiments. GAR-PINNs stops when the number of collocation points reaches the maximum, while LAR-PINNs stops when the number of collocation points reaches 5000. The numerical experiment results at this time are shown in Fig. 11. It can be seen from the Fig. 11(a), (b) that the numerical experimental results of GAR-PINNs are almost the same as those in Fig. 9(d), (e), and there is no obvious difference between them. When the number of collocation points of LAR-PINNs reaches 5000, the solution accuracy is similar to that of GLAR-PINNs.

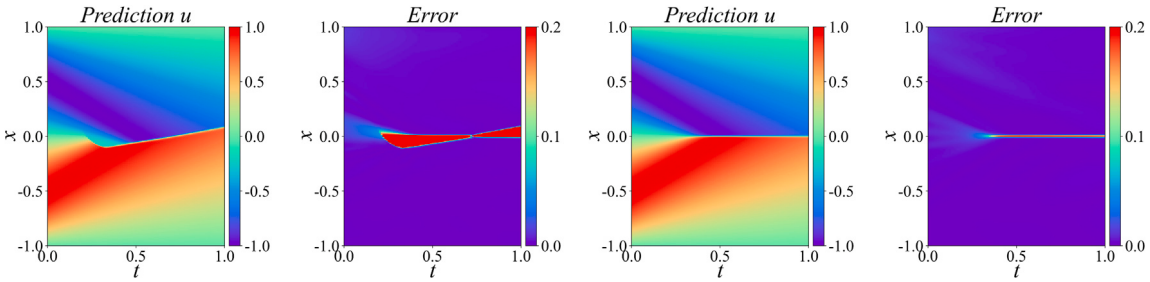
In addition, Fig. 12 shows the changes of Monte Carlo integration during the resampling process of the three methods. It can be seen from the figure that the Monte Carlo integral value of GAR-PINNs is almost unchanged. The integral value of LAR-PINNs decreased sharply when the number of residual allocation points reached 4000, and then continued to decrease with the continuous increase of residual allocation points. When the number of collocation points reaches 3000, the integral value of GLAR-PINNs drops sharply and successfully falls below the set threshold. The numerical results show that GLAR-PINNs can reasonably distribute a small number of residual collocation points under the condition of less initial residual collocation points, so as to obtain high-precision numerical solutions.

Table 4
Numerical experimental results of Burgers equation.

Methods		GLAR-PINNs	GAR-PINNs		LAR-PINNs	
Initial collocation points		100	100		100	
Final collocation points		3100	3100	10 000	3100	5100
Times		929 s	696 s	1098 s	910 s	1229 s
Absolute error	Mean	0.0037	0.0334	0.0315	0.1184	0.0039
	Std	0.0134	0.2001	0.1925	0.2174	0.0144
L_2 relative error		0.023	0.331	0.320	0.337	0.024

According to the data in Table 4, when the initial number of collocation points is set to 100, the GLAR-PINNs method achieves a L_2 relative error of 0.023 and a mean absolute error of 0.0037, requiring only 3000 resampled collocation points and a computational time of 929 s. In contrast, the LAR-PINNs method requires 5000 resampled collocation points and higher computational costs to achieve comparable accuracy to GLAR-PINNs. The GAR-PINNs method performs the worst, as even when the maximum number of collocation points 10 000 is reached, its solution accuracy does not improve significantly.

Next, we use the Burgers equation as an example to analyze the sensitivity of the initial number of collocation points $N_{T_{RC}}$ for the three methods. Under the conditions of a Monte Carlo integration threshold of $\varepsilon = 0.01$ and a maximum number of collocation points of $S = 10000$, the results are presented in Table 5. As shown in the table, as $N_{T_{RC}}$ increases, the training time of GLAR-PINNs and LAR-PINNs, as well as the number of resampled collocation points, gradually decrease. GAR-PINNs, however, only stopped training when the maximum number of collocation points reached 10 000, indicating that it



(a) GAR-PINNs prediction (b) GAR-PINNs absolute error (c) LAR-PINNs prediction (d) LAR-PINNs absolute error

Fig. 11. Predicted solutions and absolute error cloud maps of the Burgers equation using GAR-PINNs and LAR-PINNs under the same threshold ε condition. (a) GAR-PINNs prediction; (b) GAR-PINNs absolute error; (c) LAR-PINNs prediction; (d) LAR-PINNs absolute error.

Table 5
Effect of hyperparameter $N_{T_{RC}}$ on the numerical solutions of Burgers' equation: A comparative study of three experimental results.

Initial number of		GLAR-PINNs				GAR-PINNs				LAR-PINNs			
collocation points $N_{T_{RC}}$		100	500	1000	2000	100	500	1000	2000	100	500	1000	2000
Final collocation points		3100	1100	1400	2200	10 000	10 000	10 000	10 000	5100	800	1200	2100
Resampling collocation points		3000	600	400	200	9500	9500	9500	9500	5000	300	200	100
Times		929 s	147 s	144 s	90 s	1098 s	696 s	809 s	393 s	1229 s	133 s	137 s	87 s
Absolute errors	Mean	0.0037	0.0039	0.0035	0.0038	0.0315	0.0139	0.0404	0.0031	0.0039	0.0036	0.0037	0.0038
	Std	0.0134	0.0148	0.0120	0.0144	0.1925	0.0830	0.2320	0.0081	0.0144	0.0134	0.0137	0.0151
L_2 relative error		0.023	0.025	0.020	0.024	0.320	0.137	0.383	0.014	0.024	0.023	0.023	0.025

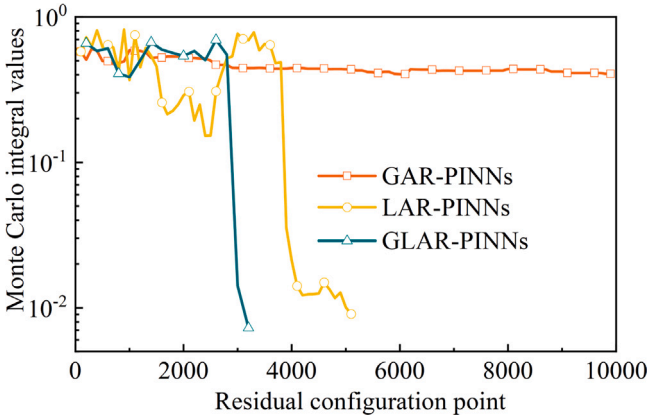


Fig. 12. Variation curve of Monte Carlo integral value in the process of resampling by three methods.

failed to meet the Monte Carlo integration threshold. Furthermore, the mean absolute error and the L_2 relative error of GLAR-PINNs and LAR-PINNs remain stable at approximately 0.003 and 0.02, respectively. In contrast, GAR-PINNs exhibit significantly larger errors. When the number of collocation points increases to 2000, the mean absolute error and L_2 relative error of GAR-PINNs improve. For LAR-PINNs, when the number of collocation points reaches 500, its solution performance becomes comparable to that of GLAR-PINNs. These observations demonstrate that all three methods depend on the initial number of collocation points, with GAR-PINNs exhibiting the strongest dependence and GLAR-PINNs the weakest. This indicates that GLAR-PINNs can achieve high-accuracy models with high computational efficiency even with a small number of collocation points, highlighting their stability. Moreover, by increasing $N_{T_{RC}}$, the training cost and the number of resampled collocation points for all three methods can be significantly reduced while maintaining relatively high solution accuracy.

3.4. Korteweg–de Vries equation

The Korteweg–de Vries equation is a type of nonlinear PDE used to describe one-dimensional shallow-water waves. It contains a third-order derivative term, which increases the complexity of solving problems using PINNs. Specifically, the propagation velocity and shape of solitary waves in the Korteweg–de Vries equation exhibit highly nonlinear characteristics in both time and space. This necessitates that PINNs possess sufficient capabilities to accurately capture these complex dynamic features, thereby effectively addressing the challenges during the solution process. The general form of the Korteweg–de Vries equation is as follows.

$$u_t + uu_x + 0.0025u_{xxx} = 0, x \in [-1, 1], t \in [0, 1], \tag{32}$$

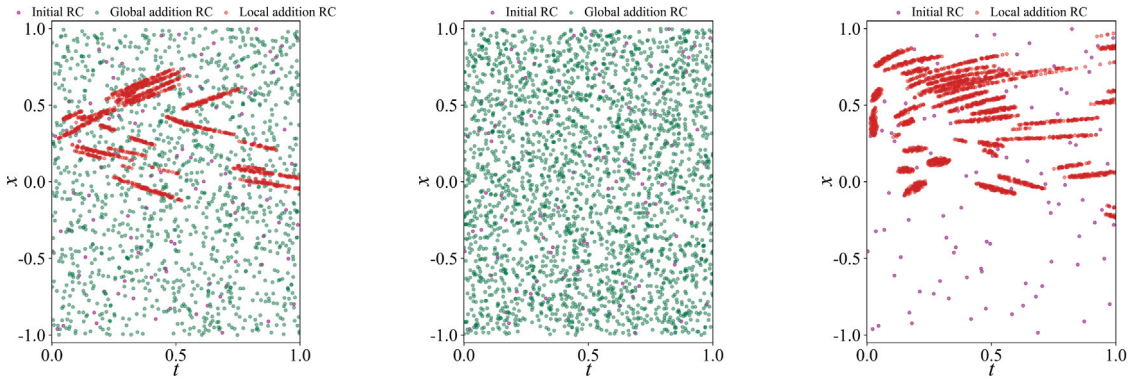
$$u(x, 0) = cos(\pi x), \tag{33}$$

$$u(-1, t) = u(1, t), \tag{34}$$

$$u_x(-1, t) = u_x(1, t) \tag{35}$$

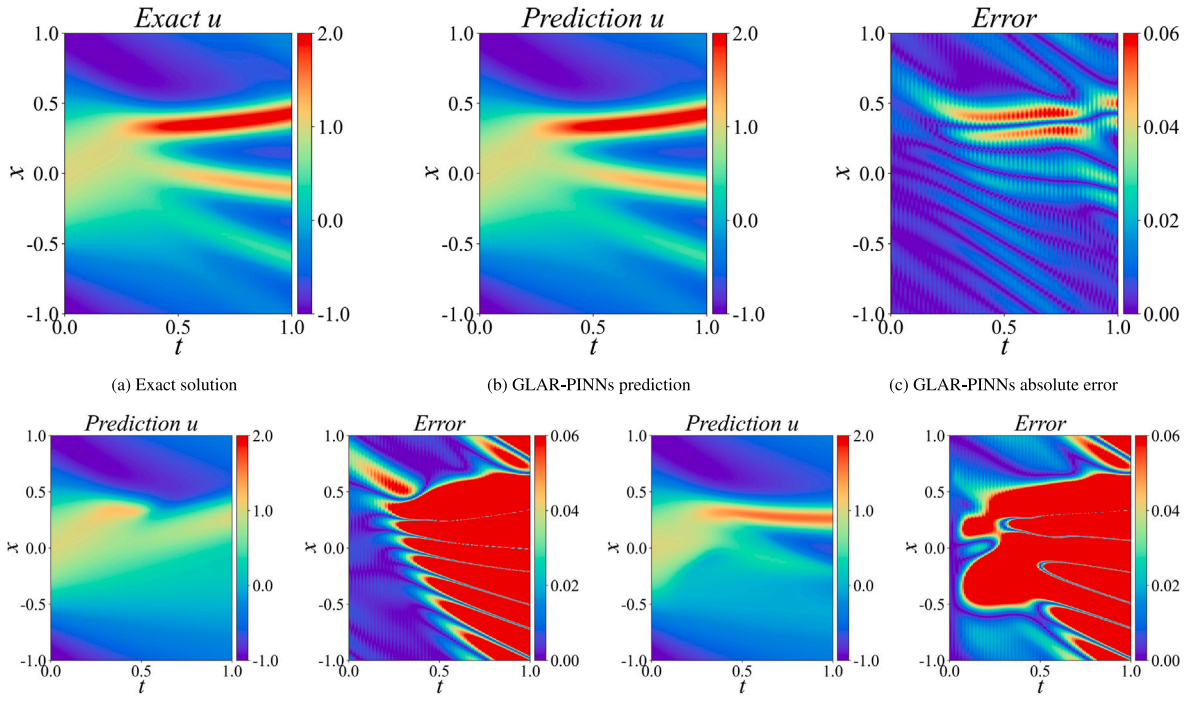
In this numerical experiment, the training data set contains 512 initial collocation points, 200 boundary condition data and 100 initial condition data. The number of collocation points used to calculate Monte Carlo integration is 50 000. The Monte Carlo integration threshold of GLAR-PINNs method is set to 0.015, and the maximum number of collocation points is limited to 10 000. In each resampling process, we add 100 allocation points in the global and local regions until the Monte Carlo integration value meets the threshold condition.

When the Monte Carlo integration value of GLAR-PINNs is lower than 0.015, 2600 collocation points are added. On this basis, we keep the Monte Carlo integration threshold unchanged, set the threshold of the maximum allocation points to 2700, and carry out the numerical experiments of GAR-PINNs and LAR-PINNs, respectively. Both methods stop training when the maximum number of allocation points is reached. The distribution of collocation points of the three methods is shown in Fig. 13. It can be seen from Fig. 13(a) and (c) that the regions with large residuals of the Korteweg–de Vries equation are



(a) GLAR-PINNs (b) GAR-PINNs (c) LAR-PINNs

Fig. 13. Final distribution of collocation points for solving the Korteweg–de Vries equation using three methods. (a) GLAR-PINNs; (b) GAR-PINNs; (c) LAR-PINNs. RC denotes the collocation points, where purple indicates the initial RC, green indicates the global addition RC, and red indicates the local addition RC.



(a) Exact solution (b) GLAR-PINNs prediction (c) GLAR-PINNs absolute error

(d) GAR-PINNs prediction (e) GAR-PINNs absolute error (f) LAR-PINNs prediction (g) LAR-PINNs absolute error

Fig. 14. Predicted solution and absolute error cloud map of the Korteweg–de Vries equation solved using three methods. (a) Exact solution; (b) GLAR-PINNs prediction; (c) GLAR-PINNs absolute error; (d) GAR-PINNs prediction; (e) GAR-PINNs absolute error; (f) LAR-PINNs prediction; (g) LAR-PINNs absolute error.

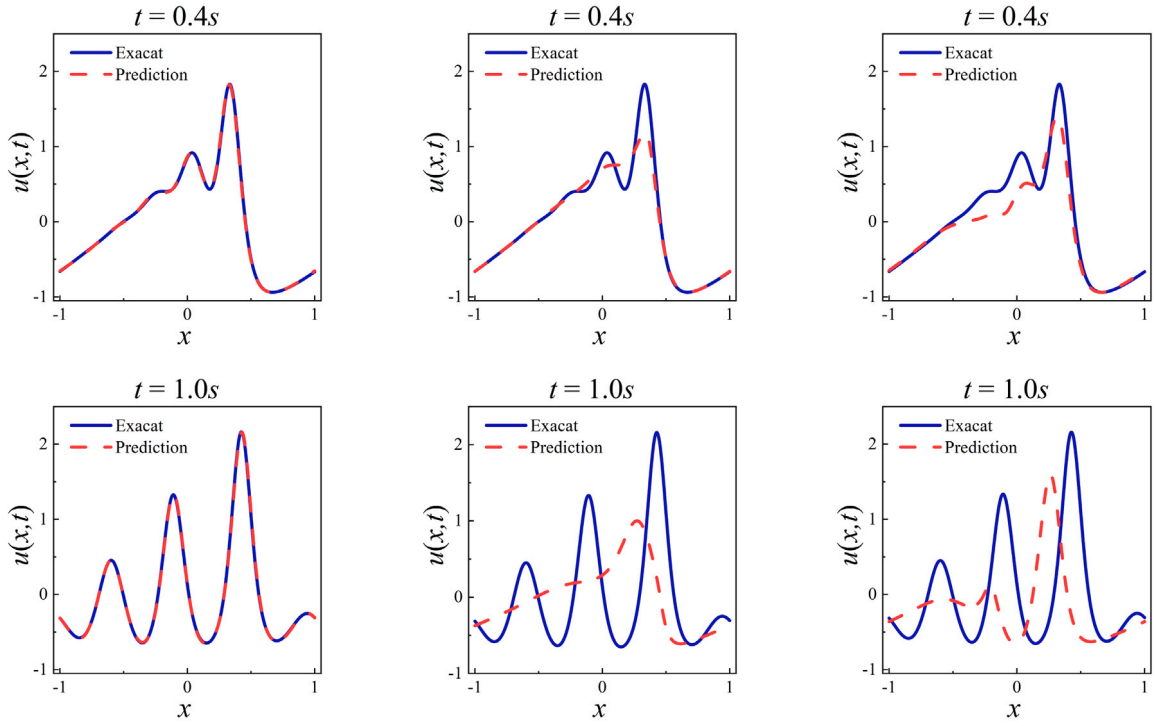
Table 6
Numerical experimental results of Korteweg–de Vries equation.

Methods		GLAR-PINNs	GAR-PINNs	LAR-PINNs
Initial collocation points		100	100	100
Final collocation points		2700	2700	2700
Times		1619 s	1281 s	2142 s
Monte Carlo integration		0.014	0.897	0.967
Absolute error	Mean	0.0099	0.2320	0.2447
	Std	0.0109	0.4361	0.4113
L_2 relative error		0.021	0.8355	0.799

mainly concentrated in the interval $[0, 1]$, which indicates that it is difficult to solve the equation using PINNs in this interval.

Fig. 14 shows the cloud diagram and error diagram of numerical results of three methods for solving Korteweg–de Vries equation. As can be seen from Fig. 14(a), (d) and (f), GAR-PINNs and LAR-PINNs are difficult to effectively capture the characteristics of Korteweg–de Vries equation, and there are large errors in the whole calculation domain. In contrast, GLAR-PINNs can effectively capture the characteristics of the equation, and the maximum absolute error is only 0.06. In Fig. 13(a), the LAR-PINNs collocation points are mainly concentrated in the upper half of $x = 0$, so the error in the lower half of Fig. 14(c) is large, which is consistent with the theoretical expectation.

In order to more intuitively show the performance of GLAR-PINNs in solving Korteweg–de Vries equation, we provide the solution results of three methods at a specific time $t = 0.4$ s, $t = 1.0$ s, as shown in Fig. 15. As can be seen from Fig. 15(a), the predicted solution of GLAR-PINNs is highly consistent with the exact solution. In contrast, there is a large deviation between the predicted solution and the exact solution of GAR-PINNs and LAR-PINNs, which is particularly obvious in Fig. 15(b), (c). According to the data in Table 6, when the 2700 collocation points of GLAR-PINNs are distributed as shown in Fig. 13(b), the Monte Carlo integral is 0.014, the L_2 relative error is 0.021, and the mean



(a) GLAR-PINNs (b) GAR-PINNs (c) LAR-PINNs

Fig. 15. Solutions of the Korteweg–de Vries equation at different times using three methods. (a) GLAR-PINNs; (b) GAR-PINNs; (c) LAR-PINNs.

Table 7
The average metrics of 5 random experiments using three methods.

Methods	Times	L_2 relative error	Absolute error	
			Mean	Std
GLAR-PINNs	1283 s	0.024	0.0112	0.0128
GAR-PINNs	1652 s	0.976	0.2720	0.5060
LAR-PINNs	2115 s	0.227	0.0826	0.1244

Table 8
Using GLAR-PINNs to solve the Korteweg–de Vries equation under different hyperparameters ε.

Threshold ε		0.010	0.015	0.020	0.030	0.040
Initial collocation points		100	100	100	100	100
Final collocation points		4900	2700	1700	2300	2700
Times		1645 s	1619 s	902 s	1449 s	1117 s
Absolute error	Mean	0.0120	0.0099	0.0114	0.0221	0.0125
	Std	0.0138	0.0109	0.0140	0.0342	0.0137
L_2 relative error		0.026	0.021	0.026	0.058	0.026

absolute error is 0.0099, which is significantly better than GAR-PINNs and LAR-PINNs in solving accuracy. In addition, the solving efficiency of GLAR-PINNs is only 0.75 times that of LAR-PINNs.

Next, under the same conditions, we conducted five randomized numerical experiments using the three methods for the Korteweg–de Vries equation to evaluate the stability of GLAR-PINNs. The numerical results are shown in Fig. 16 and Table 7. As can be seen from Fig. 16, the L_2 relative error and mean absolute error of GLAR-PINNs exhibit nearly linear trends across the five experiments, demonstrating its excellent stability. The mean and standard deviation of the mean absolute error from the five experimental results in Table 7 further confirm this conclusion. By combining the results from Fig. 16 and Table 7, it is evident that GAR-PINNs and LAR-PINNs exhibit larger solution errors, poorer stability, and require higher computational costs. The numerical results fully demonstrate that, with a small number of initial residual collocation points, GLAR-PINNs can achieve high PINNs model accuracy with relatively low computational costs while maintaining stability.

A sensitivity analysis was conducted on the hyperparameter ε of GLAR-PINNs using the Korteweg–de Vries equation, with numerical experiment results presented in Table 8. The observations from the table indicate that as the Monte Carlo integration threshold ε increases, both the number of resampled collocation points and the computational cost of GLAR-PINNs exhibit a trend of initial increase followed by a decrease. In the interval $\varepsilon \in [0.01, 0.02]$, the L_2 relative error and mean absolute error remain stable at 0.02 and 0.01, respectively. In the interval $\varepsilon \in [0.02, 0.04]$, the L_2 relative error and mean absolute error exhibit an initial increase followed by a subsequent decrease. These findings suggest that the hyperparameter ε not only significantly influences the computational efficiency of GLAR-PINNs but also plays a critical role in determining the model's accuracy. Therefore, choosing the appropriate Monte-Carlo integral threshold ε for different problems can not only reduce the calculation cost, but also obtain higher model accuracy.

3.5. Allen–Cahn equation

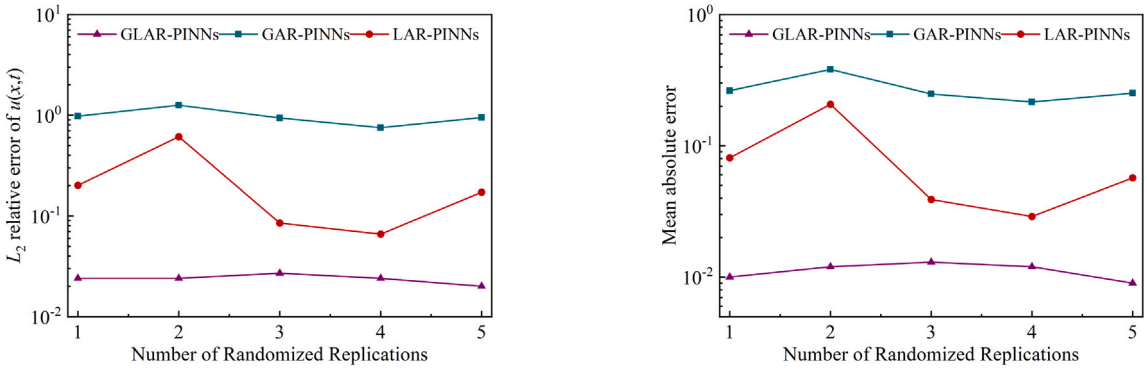
Next, we will study a class of widely used nonlinear Allen–Cahn equation, which is often used to describe physical phenomena such as phase transition and interface dynamics. The specific form is as follows.

$$u_t - Du_{xx} - 5(u - u^3) = 0, x \in [-1, 1], t \in [0, 1], \tag{36}$$

$$u(x, 0) = x^2 cos(\pi x), \tag{37}$$

$$u(-1, t) = u(1, t) = -1 \tag{38}$$

In this experiment, the parameter D is set to 1. The training data set contains 201 initial condition data, 200 boundary condition data and



(a) L_2 relative error (b) Mean absolute error

Fig. 16. Comparison of error trends for solving the Korteweg–de Vries equation using three methods in five random experiments. (a) L_2 relative error; (b) Mean absolute error.

Table 9
Numerical experimental results of Allen–Cahn equation.

Methods	Threshold ϵ	Initial collocation points	Final collocation points	Times	L_2 relative error	Absolute error	
						Mean	Std
GLAR-PINNs	0.002	100	700	190 s	0.0174	0.0009	0.0016
GAR-PINNs	0.002	100	1500	252 s	0.0053	0.0007	0.0010
LAR-PINNs	0.002	100	1800	360 s	0.1244	0.0010	0.0017

Table 10
Using GLAR-PINNs to solve the Allen–Cahn under different hyperparameters S.

The maximum collocation points number S	Initial collocation points	Times	Monte Carlo integration	L_2 relative error	Absolute error	
					Mean	Std
300	100	119 s	0.0036	0.0341	0.0017	0.0023
500	100	149 s	0.0030	0.0058	0.0018	0.0030
700	100	190 s	0.0017	0.0174	0.0010	0.0015
900	100	192 s	0.0017	0.0294	0.0010	0.0018
1100	100	220 s	0.0017	0.0409	0.0009	0.0015

100 initial collocation points. The number of collocation points used to calculate Monte Carlo integration is 25 000. The threshold value of Monte Carlo integration is set to 0.002, and other parameter settings are similar to Korteweg–de Vries equation. Table 9 shows the numerical experimental results of solving Allen–Cahn equation by GLAR-PINNs, GAR-PINNs and LAR-PINNs. As can be seen from the table, GLAR-PINNs require the smallest number of final collocation points and exhibit the highest computational efficiency. In terms of accuracy, although the L_2 relative error and mean absolute error for GAR-PINNs are lower than those of GLAR-PINNs, the L_2 relative error of GAR-PINNs is only 32% of that of GLAR-PINNs, showing no significant difference at the 10^{-2} magnitude level. However, the L_2 relative error of LAR-PINNs is 7.14 times that of GLAR-PINNs, indicating the poorest performance among the methods.

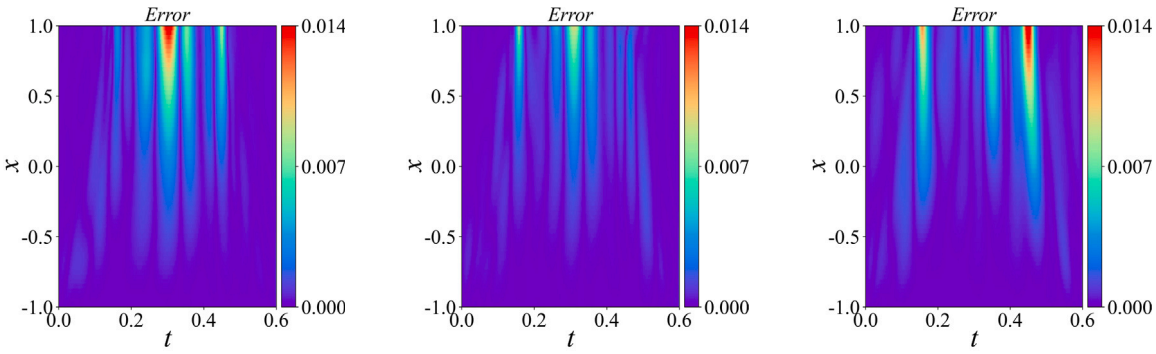
Next, we will use GLAR-PINNs to solve the Allen–Cahn equation as an example and conduct a sensitivity analysis on the hyperparameter S, which represents the maximum number of collocation points. According to the numerical experimental results in Table 10, it can be observed that as the value of S increases, the computational cost of GLAR-PINNs correspondingly increases, while the Monte Carlo integral value shows a trend of initially decreasing and then stabilizing. Meanwhile, the L_2 relative error exhibits a characteristic of initially decreasing and then gradually increasing. The variation of GLAR-PINNs solution precision shows that adding too many collocation points will affect the solution performance of PINNs and make it fall into a local optimal state. In order to improve the solving efficiency, avoid falling into the local optimal phenomenon, and ensure the good and stable performance of GLAR-PINNs, we need to choose the appropriate hyperparameter S according to different problems.

The absolute error cloud chart in Fig. 17 shows that in the region near $t = 0.3$ s, the error of Allen–Cahn equation is relatively large. However, the overall absolute error of GLAR-PINNs is still in the order of 10^{-3}, showing good accuracy. Fig. 18 shows the loss function and Monte Carlo integral curves of the three methods during the training process. Fig. 18(a) shows that the loss function values of the three methods can be reduced to 10^{-5}, which shows that good optimization effects are achieved in the process of neural network training. While Fig. 18(b) shows that during the training process, GLAR-PINNs can rapidly improve the training accuracy of the model with fewer collocation points.

Although GAR-PINNs showed the highest accuracy in Allen–Cahn's numerical experiments, the L_2 relative error of GAR-PINNs is only 32% of that of GLAR-PINNs, which is not a significant difference on the order of 10^{-2}. Moreover, GLAR-PINNs exhibit the highest efficiency. In summary, a comparative analysis of the three methods used to solve the Allen–Cahn equation reveals that GLAR-PINNs can achieve a high level of efficiency and accuracy in obtaining the numerical solution by employing a relatively small number of collocation points, which are strategically distributed throughout the solution domain.

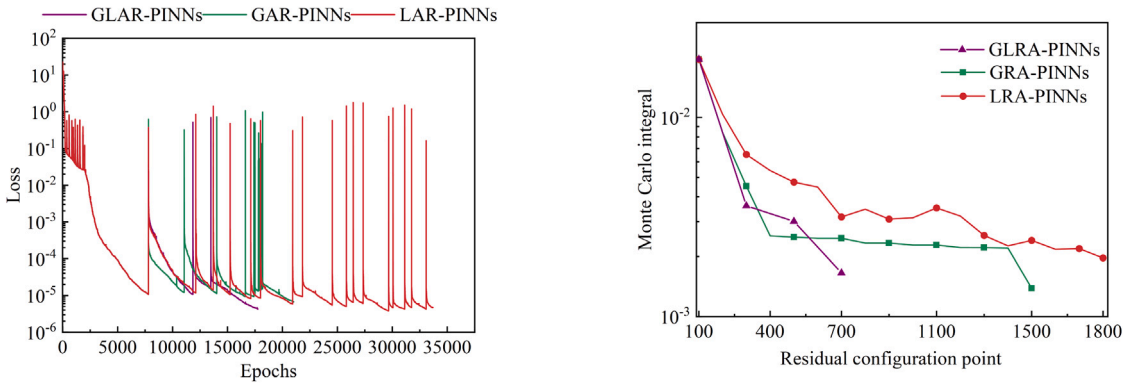
3.6. Navier–Stokes equation

Through the ablation comparison experiments of linear and nonlinear PDEs, we have demonstrated the effectiveness and generalization ability of GLAR-PINNs, achieving relatively high solution accuracy. Next, we will employ GLAR-PINNs to reconstruct the flow around a two-dimensional triangular cylinder, thereby verifying its practical applicability in engineering fields. Efficient and accurate reconstruction of the flow field information around the cylinder can provide deep



(a) GLAR-PINNs (b) GAR-PINNs (c) LAR-PINNs

Fig. 17. Absolute errors cloud map of the Allen–Cahn equation solved using three methods. (a) GLAR-PINNs; (b) GAR-PINNs; (c) LAR-PINNs.



(a) Loss function (b) Monte Carlo integral

Fig. 18. Evolution of loss function and Monte Carlo integral value during solving Allen–Cahn equation by three methods. (a) Loss function; (b) Monte Carlo integral.

insights into the vortex street, pressure, and velocity distributions in the flow around the cylinder. This is of great significance for the optimization design in aerospace, wind power generation, and pipeline flow applications. The governing equation for the flow around a cylinder is the Navier–Stokes equation, which is both nonlinear and coupled. We consider the general form of the incompressible two-dimensional Navier–Stokes equations as follows.

$$u_t + uu_x + vu_y = -\frac{1}{\rho}p_x + \frac{1}{Re}(u_{xx} + u_{yy}), \tag{39}$$

$$v_t + uv_x + vv_y = -\frac{1}{\rho}p_y + \frac{1}{Re}(v_{xx} + v_{yy}), \tag{40}$$

$$u_x + v_y = 0 \tag{41}$$

The residual function is

$$R_u(x, y, t; \theta) = u_t + uu_x + vu_y + \frac{1}{\rho}p_x - \frac{1}{Re}(u_{xx} + u_{yy}), \tag{42}$$

$$R_v(x, y, t; \theta) = v_t + uv_x + vv_y + \frac{1}{\rho}p_y - \frac{1}{Re}(v_{xx} + v_{yy}), \tag{43}$$

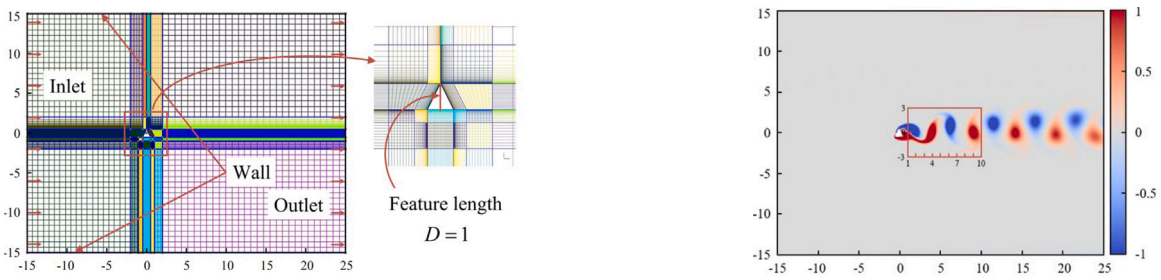
$$R_m(x, y, t; \theta) = u_x + v_y \tag{44}$$

The high fidelity data set of the flow around a triangular cylinder is obtained through the spectral/hp element solver NekTar. Fig. 19(a) shows the meshing of the solution domain, while Fig. 19(b) shows the flow field data of the flow around a triangular cylinder in the selected $x \in [1, 10]$ and $y \in [-3, 3]$ regions, which will be used as the reference data, boundary conditions and initial conditions data set. In this experiment, the flow velocity $u = 1$, dynamic viscosity coefficient $\nu = 0.01$, fluid density $\rho = 1$, Reynolds number $Re = u\rho D/\nu = 100$, where $D = 1$ is the characteristic length of the triangular cylinder.

The network structure of GLAR-PINNs contains three inputs (x, y, t) and three outputs (u, v, p). The training data set includes 2000 boundary conditions, 1000 initial conditions and 500 initial collocation points. The collocation points for calculating the Monte Carlo integration are 200 000, the threshold is 0.7 (the Monte Carlo integration of residual R_u is used as the benchmark for evaluating the accuracy of the model), and the maximum allocation point is 20 000. In the training process of GLAR-PINNs, according to the Monte Carlo integration threshold, 300 collocation points are randomly added globally each time, and 100 points with large residual errors of R_u, R_v and R_m are selected to be added locally.

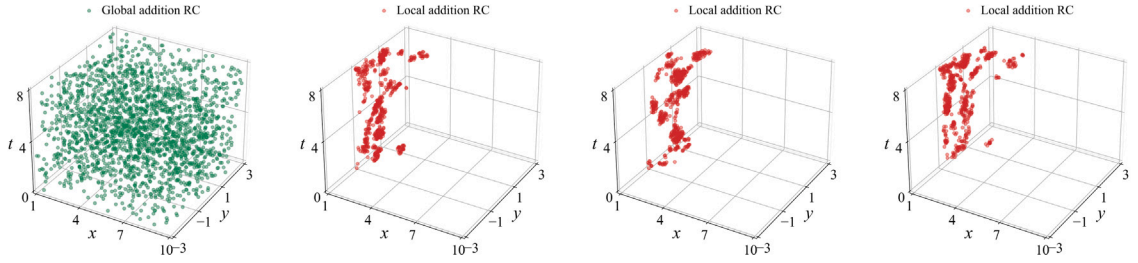
In the process of reconstructing the flow around a two-dimensional triangular cylinder by GLAR-PINNs, when the Monte Carlo integral value of R_u residuals is lower than 0.7, the whole process takes 3115 s, and 8400 collocation points are added in the global and local regions. The final distribution of collocation points is shown in Fig. 20. It can be seen from the figure that the regions with large residuals of R_u, R_v and R_m are mainly concentrated in the range of $x \in [1, 4]$ and $y \in [-1, 1]$. This is mainly because the fluid characteristics behind the cylinder are complex, and it is difficult for PINNs to capture its flow characteristics, so the residual values in these areas are large.

The numerical experimental results are shown in Fig. 21. It can be seen from the figure that the predicted results of velocity u, v, pressure p and vorticity w in two-dimensional flow field have better consistency compared with the accurate results. However, observing the relative error cloud map shows that the error ranges of velocity u, v and pressure p gradually increase over time. Secondly, the relative error of vorticity w is mainly concentrated in the vortex region. It can be seen from Table 11 that the L_2 relative error of vorticity w is the largest, while the L_2 relative error of velocity u is the smallest, and the solution accuracy is the highest. Through the observation of the



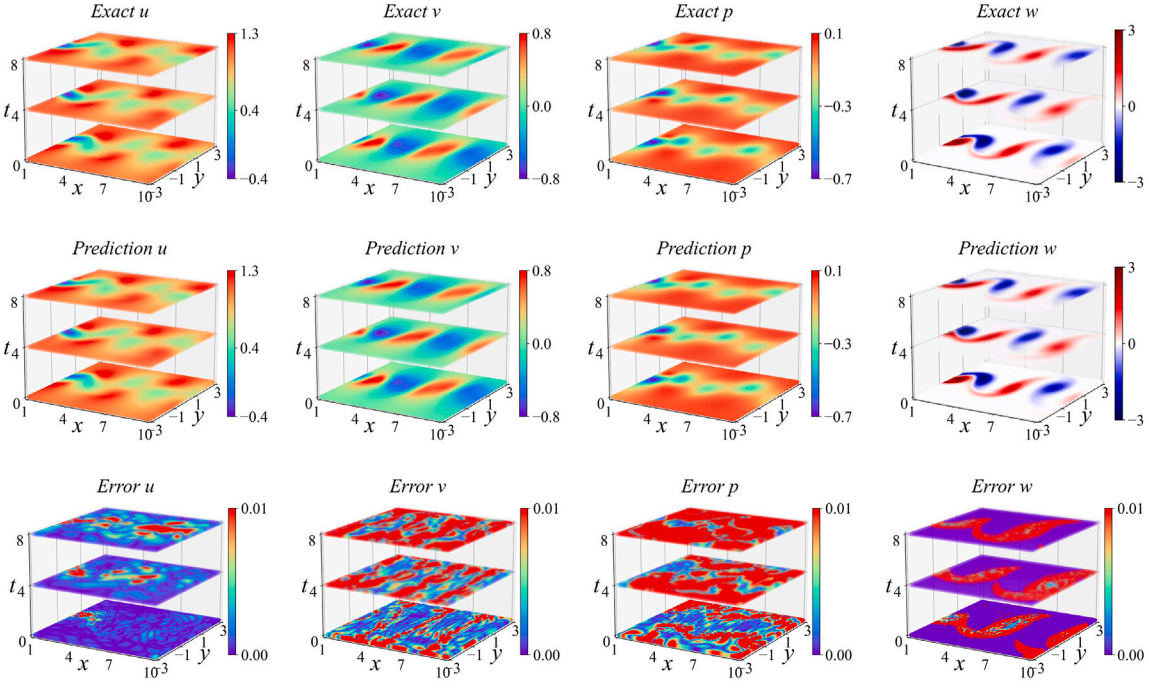
(a) Mesh (b) Data set

Fig. 19. Computational grids and extracted data sets for flows around triangular cylinders. (a) Mesh; (b) Data set.



(a) Add RC globally (b) Add RC locally based on R_u (c) Add RC locally based on R_v (d) Add RC locally based on R_m

Fig. 20. GLAR-PINNs adds the final distribution of collocation points in the global and local regions. RC denotes the collocation points, where green indicates the global addition RC, and red indicates the local addition RC. (a) Add RC globally; (b) Add RC locally based on R_u; (c) Add RC locally based on R_v; (d) Add RC locally based on R_m.



(a) Velocity u (b) Velocity v (c) Pressure p (d) Vorticity w

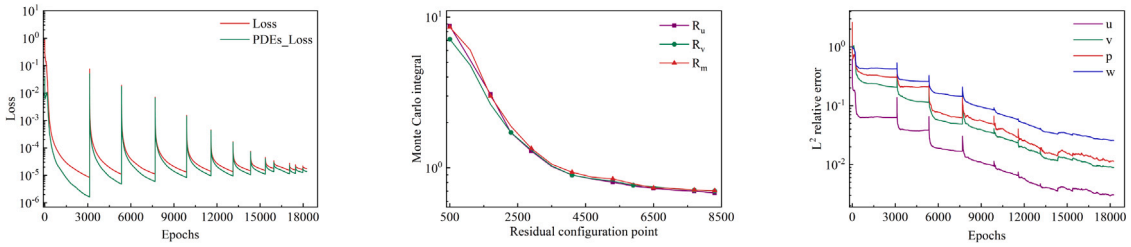
Fig. 21. GLAR-PINNs reconstruction of flow field information and error plot around a two-dimensional triangular cylinder. (a) Velocity u; (b) Velocity v; (c) Pressure p; (d) Vorticity w.

average relative error at different times, we still find that the error of vorticity is always the largest relative to the velocity u and v, mainly because the vorticity is calculated by the derivative of the velocity u and v, and the velocity u and v themselves have certain errors, so there is error accumulation, resulting in relatively large errors of vorticity. In addition, the average relative errors of velocity u, v, pressure p and vorticity w gradually increase over time, which is also consistent with the results in Fig. 21.

Fig. 22 shows the variation curves of the loss function, Monte Carlo integration, and L_2 relative error during the training process. Fig. 22(a) shows that during the training process, both the overall loss and PDEs residual loss can reach the order of magnitude of 10^{-5}, and their trends

Table 11
Numerical results of the reconstruction of the flow field around a triangular cylinder using GLAR-PINNs.

	Times	Monte Carlo integration	L_2 relative error	Relative error					
				$t = 0$ s		$t = 4$ s		$t = 8$ s	
				Mean	Std	Mean	Std	Mean	Std
u	3115 s	0.68	0.0030	0.0015	0.015	0.0051	0.182	0.0056	0.103
v		0.70	0.0089	0.0206	0.604	0.0689	1.587	0.0822	3.402
p		0.70	0.0114	0.0367	0.316	0.0691	0.631	0.1900	5.137
w		/	0.0258	2.3620	23.793	2.682	28.472	3.6081	16.183



(a) Loss function (b) Monte Carlo integration (c) L_2 relative error

Fig. 22. Evolution curves of loss function, Monte Carlo integral value and L_2 relative error in the training process of reconstructing the flow around a triangular cylinder by GLAR-PINNs. (a) Overall loss and PDEs residual loss curves; (b) Monte Carlo integration versus collocation points variation curve; (c) L_2 relative error curve for predicting velocity, pressure, and vorticity.

are similar. Fig. 22(b) reveals that during the training process of GLAR-PINNs, the Monte Carlo integration trends of residuals R_u, R_v and R_m are consistent, and the numerical differences between the three are minimal. Fig. 22(c) shows that in the two-dimensional triangular cylinder flow field reconstructed by GLAR-PINNs, the accuracy of the horizontal velocity u is the highest, while the accuracy of the vorticity w is relatively low. However, with the distribution and changes of collocation points, the accuracy of reconstructed flow field information shows a gradually improving trend.

In summary, the GLAR-PINNs method proposed in this article can effectively reconstruct the flow field characteristics of two-dimensional cylinder flow. This method can add collocation points in the appropriate solution area according to the complexity of the flow field characteristics, so that GLAR-PINNs can accurately capture the flow field characteristics of the complex area around the two-dimensional cylinder, thereby improving the overall accuracy of flow field information reconstruction.

4. Conclusions

Addressing the issues of existing collocation points adaptive resampling methods that focus only on areas with high local residuals and are overly sensitive to the number of initial collocation points, this paper proposes a global–local adaptive resampling Physics-informed neural networks (GLAR-PINNs). To demonstrate the effectiveness of our proposed GLAR-PINNs method, ablation experiments were conducted on five partial differential equations (including two linear and three nonlinear equations). The numerical experimental results for the linear partial differential equations show that while GAR-PINNs achieve the highest solution accuracy, the accuracy difference between GLAR-PINNs and GAR-PINNs is only within the order of 10^{-3} to 10^{-2}, with LAR-PINNs performing the worst. The numerical experiments for the nonlinear PDEs indicate that GLAR-PINNs can achieve an L_2 relative error within the order of 10^{-2} under conditions of only 100 initial collocation points, enhancing the solution accuracy by one order of magnitude compared to GAR-PINNs and LAR-PINNs. Finally, we employed the proposed GLAR-PINNs to reconstruct the flow field around a two-dimensional triangular cylinder. The numerical experimental results show that under conditions of 500 initial collocation points, the method achieves an L_2 relative error within the order of 10^{-2} for the flow field information, demonstrating good solution accuracy.

Based on the numerical experiments conducted, we can draw the following conclusions:

1. GLAR-PINNs achieve high accuracy in solving both linear and nonlinear PDEs, demonstrating good generalization capabilities and robustness.
2. GLAR-PINNs address the issue of adaptive collocation points sampling strategies that focus only on local areas with high residuals while neglecting the global quantity and distribution of collocation points.
3. GLAR-PINNs can obtain high-accuracy numerical solutions for PDEs even under conditions of a very limited number of initial collocation points, effectively resolving the dependency on an appropriate number of initial collocation points.
4. During the training process, GLAR-PINNs can assess model accuracy based on Monte Carlo integration and adaptively add additional collocation points in both global and local regions to optimize the model's solution accuracy.

In conclusion, the GLAR-PINNs proposed in this paper significantly improves the accuracy of PINNs in solving complex nonlinear and higher-order PDEs under the condition of very few collocation points, and provides a theoretical basis for the application and development of artificial intelligence in engineering. However, through hyperparameter sensitivity analysis of different types of PDEs equations, we found that different thresholds and the number of initial collocation points have significant effects on the solving efficiency and accuracy of GLAR-PINNs, and may even cause PINNs to fall into local optimality during the solving process. Therefore, in the following research work, we will continue to focus on GLAR-PINNs, so that it can adapt to adjust the threshold or the number of initial collocation points when facing different types of PDEs, so as to improve the solving efficiency and accuracy of GLAR-PINNs.

CRediT authorship contribution statement

Lei Gao: Writing – review & editing, Writing – original draft, Visualization, Validation, Software, Methodology, Investigation, Formal analysis, Data curation, Conceptualization. **Dan Zhang:** Writing – review & editing, Supervision, Resources, Methodology. **Yaoran Chen:**

Writing – review & editing, Methodology, Funding acquisition. **Xiaowei Li:** Supervision, Methodology, Investigation. **Chunxin Li:** Supervision, Methodology, Investigation. **Yan Peng:** Supervision, Resources, Methodology, Funding acquisition.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

The financial support from the National Natural Science Foundation of China (Nos. 62225308, 42306226 and 62073332) and the Program of the Pujiang National Laboratory (No. P22KN00391) is gratefully acknowledged. The financial support from Shanghai University (No. 13-G210-23-368) is also acknowledged.

Data availability

Data will be made available on request.

References

Baldan, M., Barba, Paolo Di, Lowther, D.A., 2023. Physics-informed neural networks for inverse electromagnetic problems. IEEE Trans. Magn. 59, 1–5. http://dx.doi.org/10.1109/tmag.2023.3247023.

Cao, G., Wang, X., 2025. Parametric extended physics-informed neural networks for solid mechanics with complex mixed boundary conditions. J. Mech. Phys. Solids 194, 105944. http://dx.doi.org/10.1016/j.jmps.2024.105944.

Chen, Y., Koohy, S., 2024. GPT-PINN: Generative pre-trained physics-informed neural networks toward non-intrusive meta-learning of parametric PDEs. Finite Elements Anal. Des. 228, http://dx.doi.org/10.1016/j.finel.2023.104047, 104047–104047.

Chen, Y., Lu, L., Karniadakis, G.E., Dal Negro, L., 2020. Physics-informed neural networks for inverse problems in nano-optics and metamaterials. Opt. Express 28, 11618. http://dx.doi.org/10.1364/oe.384875.

Cho, W., Jo, M., Lim, H., Lee, K., Lee, D., Hong, S., Park, N., 2024. Parameterized physics-informed neural networks for parameterized PDEs, ArXiv (Cornell University). http://dx.doi.org/10.48550/arxiv.2408.09446.

Eivazi, Hamidreza, Tahani, Mojtaba, Schlatter, P., Vinuesa, R., 2022. Physics-informed neural networks for solving Reynolds-averaged Navier–Stokes equations. Phys. Fluids 34, http://dx.doi.org/10.1063/5.0095270.

Fan, J., Duan, W., Huang, L., Zhang, L., Yang, K., 2023. High-fidelity flow field reconstruction model for incompressible fluid with physical constraints. Ocean. Eng. 280, http://dx.doi.org/10.1016/j.oceaneng.2023.114597, 114597–114597.

Francois, J., Laubscher, R., 2023. Evaluation of physics-informed neural network solution accuracy and efficiency for modeling aortic transvalvular blood flow. Math. Comput. Appl. 28, http://dx.doi.org/10.3390/mca28020062, 62–62.

Gao, Z., Tang, T., Yan, L., Zhou, T., 2023a. Failure-informed adaptive sampling for PINNs, part II: Combining with re-sampling and subset simulation. Commun. Appl. Math. Comput. 6, 1720–1741. http://dx.doi.org/10.1007/s42967-023-00312-7.

Gao, Z., Yan, L., Zhou, T., 2023b. Failure-informed adaptive sampling for PINNs. SIAM J. Sci. Comput. 45, A1971–A1994. http://dx.doi.org/10.1137/22m1527763.

Guo, L., Wu, H., Yu, X., Zhou, T., 2022. Monte Carlo fPINNs: Deep learning method for forward and inverse problems involving high dimensional fractional partial differential equations. Comput. Methods Appl. Mech. Eng. 400, 115523. http://dx.doi.org/10.1016/j.cma.2022.115523.

Guo, J., Yao, Y., Wang, H., Gu, T., 2023. Pre-training strategy for solving evolution equations based on physics-informed neural networks. J. Comput. Phys. 489, http://dx.doi.org/10.1016/j.jcp.2023.112258, 112258–112258.

Hou, X., Zhou, X., Liu, Y., 2024. Reconstruction of ship propeller wake field based on self-adaptive loss balanced physics-informed neural networks. Ocean. Eng. 309, http://dx.doi.org/10.1016/j.oceaneng.2024.118341, 118341–118341.

Huang, Y., Liu, H., Zhao, Y., Fei, M., 2024. Chebyshev spectral approximation-based physics-informed neural network for solving higher-order nonlinear differential equations. Eng. Comput. http://dx.doi.org/10.1007/s00366-024-02073-0.

Jiao, Y., Li, D., Lu, X., Yang, J.Z., Yuan, C., 2024. A Gaussian mixture distribution-based adaptive sampling method for physics-informed neural networks. Eng. Appl. Artif. Intell. 135, http://dx.doi.org/10.1016/j.engappai.2024.108770, 108770–108770.

Jin, X., Cai, S., Li, H., Karniadakis, G.E., 2020. NSFnets (Navier–Stokes flow nets): Physics-informed neural networks for the incompressible Navier–Stokes equations. J. Comput. Phys. 109951. http://dx.doi.org/10.1016/j.jcp.2020.109951.

Joshi, A., Papados, A., Kumar, R., 2023. Investigation of low and high-speed fluid dynamics problems using physics-informed neural network. Int. J. Comput. Fluid Dyn. 37, 149–166. http://dx.doi.org/10.1080/10618562.2023.2285330.

Kumar, A.K., Jain, S., Jain, S., Ritam, M., Xia, Y., Chandra, R., 2023. Physics-informed neural entangled-ladder network for inhalation impedance of the respiratory system. Comput. Methods Programs Biomed. 231, 107421. http://dx.doi.org/10.1016/j.cmpb.2023.107421.

Leonenko, N., Podlubny, I., 2022a. Monte Carlo method for fractional-order differentiation. Fract. Calc. Appl. Anal. 25, 346–361. http://dx.doi.org/10.1007/s13540-022-00017-3.

Leonenko, N., Podlubny, I., 2022b. Monte Carlo method for fractional-order differentiation extended to higher orders. Fract. Calc. Appl. Anal. 25, 841–857. http://dx.doi.org/10.1007/s13540-022-00048-w.

Li, Y., Wan, D., Wang, Z., Hu, D., 2024. Physics-constrained deep learning approach for solving inverse problems in composite laminated plates. Compos. Struct. 348, http://dx.doi.org/10.1016/j.compstruct.2024.118514, 118514–118514.

Lin, S., Chen, Y., 2024. The improved backward compatible physics-informed neural networks for reducing error accumulation and applications in data-driven higher-order rogue waves. Chaos: Interdiscip. J. Nonlinear Sci. 34, http://dx.doi.org/10.1063/5.0191283.

Liu, Y., Chen, L., Ding, J., Chen, Y., 2024. An adaptive sampling method based on expected improvement function and residual gradient in PINNs. IEEE Access. 12, 92130–92141. http://dx.doi.org/10.1109/access.2024.3422224.

Lu, L., Meng, X., Mao, Z., Karniadakis, G.E., 2021. DeepXDE: A deep learning library for solving differential equations. SIAM Rev. 63, 208–228. http://dx.doi.org/10.1137/19m1274067.

Mao, z., Jagtap, A.D., Karniadakis, G.E., 2020. Physics-informed neural networks for high-speed flows. Comput. Methods Appl. Mech. Eng. 360, 112789. http://dx.doi.org/10.1016/j.cma.2019.112789.

Mattey, R., Ghosh, S., 2022. A novel sequential method to train physics informed neural networks for Allen Cahn and Cahn Hilliard equations. Comput. Methods Appl. Mech. Eng. 390, http://dx.doi.org/10.1016/j.cma.2021.114474, 114474–114474.

Maziar Raissi, Perdikaris, P., George Em Karniadakis, 2017. Physics informed deep learning (part II): Data-driven discovery of nonlinear partial differential equations, ArXiv (Cornell University). http://dx.doi.org/10.48550/arxiv.1711.10566.

Nabian, M.A., Gladstone, R.J., Meidani, H., 2021. Efficient training of physics-informed neural networks via importance sampling. Comput.-Aided Civ. Infrastruct. Eng. http://dx.doi.org/10.1111/mice.12685.

Raissi, M., Karniadakis, G.E., 2018. Hidden physics models: Machine learning of nonlinear partial differential equations. J. Comput. Phys. 357, 125–141. http://dx.doi.org/10.1016/j.jcp.2017.11.039.

Raissi, M., Perdikaris, P., Karniadakis, G.E., 2017a. Inferring solutions of differential equations using noisy multi-fidelity data. J. Comput. Phys. 335, 736–746. http://dx.doi.org/10.1016/j.jcp.2017.01.060.

Raissi, M., Perdikaris, P., Karniadakis, G.E., 2017b. Machine learning of linear differential equations using Gaussian processes. J. Comput. Phys. 348, 683–693. http://dx.doi.org/10.1016/j.jcp.2017.07.050.

Raissi, M., Perdikaris, P., Karniadakis, G.E., 2017c. Physics informed deep learning (part I): Data-driven solutions of nonlinear partial differential equations. http://dx.doi.org/10.48550/arXiv.1711.10561, ArXiv.org..

Raissi, M., Perdikaris, P., Karniadakis, G.E., 2018. Numerical Gaussian processes for time-dependent and nonlinear partial differential equations. SIAM J. Sci. Comput. 40, A172–A198. http://dx.doi.org/10.1137/17m1120762.

Raissi, M., Perdikaris, P., Karniadakis, G.E., 2019. Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. J. Comput. Phys. 378, 686–707. http://dx.doi.org/10.1016/j.jcp.2018.10.045.

Rouhollahi, Amir, Rismanian, Milad, Ebrahimi, A., Ilegbusi, O.J., Nezami, F.R., 2024. Prediction of directional solidification in freeze casting of biomaterial scaffolds using physics-informed neural networks. Biomed. Phys. Eng. Express 10, http://dx.doi.org/10.1088/2057-1976/ad7960, 065023–065023.

Subramanian, S., Kirby, R.M., Mahoney, M.W., Gholami, A., 2023. Adaptive self-supervision algorithms for physics-informed neural networks. Front. Artif. Intell. Appl. http://dx.doi.org/10.3233/faia230521.

Tang, K., Wan, X., Yang, C., 2023. DAS-PINNs: A deep adaptive sampling method for solving high-dimensional partial differential equations. J. Comput. Phys. 476, 111868. http://dx.doi.org/10.1016/j.jcp.2022.111868.

Visser, C., Heinlein, A., Giovanardi, B., 2024. PACMANN: Point adaptive collocation method for artificial neural a) networks, ArXiv (Cornell University). http://dx.doi.org/10.48550/arxiv.2411.19632.

W, C.L., Zhao, J., 2021. Solving Allen-Cahn and Cahn-Hilliard equations using the adaptive physics informed neural networks. Commun. Comput. Phys. 29, 930–954. http://dx.doi.org/10.4208/cicp.oa-2020-0086.

Wang, S., Karniadakis, G.E., 2024. GMC-PINNs: A new general Monte Carlo PINNs method for solving fractional partial differential equations on irregular domains. Comput. Methods Appl. Mech. Eng. 429, http://dx.doi.org/10.1016/j.cma.2024.117189, 117189–117189.

Wang, L., Liu, G., Wang, G., Zhang, K., 2024. M-PINN: A mesh-based physics-informed neural network for linear elastic problems in solid mechanics. Int. J. Numer. Methods Eng. 125, http://dx.doi.org/10.1002/nme.7444.

Wu, C., Zhu, M., Tan, Q., Kartha, Y., Lu, L., 2023. A comprehensive study of non-adaptive and residual-based adaptive sampling for physics-informed neural networks. Comput. Methods Appl. Mech. Eng. 403, 115671. http://dx.doi.org/10.1016/j.cma.2022.115671.

Xiang, Z., Peng, W., Yao, W., Liu, X., Zhang, X., 2025. Physics-informed neural implicit flow neural network for parametric PDEs. Neural Netw. 185, 107166. http://dx.doi.org/10.1016/j.neunet.2025.107166.

Xiao, Y., Yang, L.M., Shu, C., Dong, H., Du, Y.J., Song, Y.X., 2024. Least-square finite difference-based physics-informed neural network for steady incompressible flows. Comput. Math. Appl. 175, 33–48. http://dx.doi.org/10.1016/j.camwa.2024.08.035.

Yang, Y., Yang, Q., Deng, Y., He, Q., 2024. Moving sampling physics-informed neural networks induced by moving mesh PDE. Neural Netw. 180, 106706. http://dx.doi.org/10.1016/j.neunet.2024.106706.

Yao, B., Wang, Z., Fang, Z., Li, Z., 2024. Reconstruction of downburst wind fields using physics-informed neural network. J. Wind. Eng. Ind. Aerodyn. 254, http://dx.doi.org/10.1016/j.jweia.2024.105935, 105935–105935.

Yin, M., Zheng, X., Humphrey, J.D., Karniadakis, G.E., 2021. Non-invasive inference of thrombus material properties with physics-informed neural networks. Comput. Methods Appl. Mech. Eng. 375, 113603. http://dx.doi.org/10.1016/j.cma.2020.113603.

Yu, J., Lu, L., Meng, X., Karniadakis, G.E., 2022. Gradient-enhanced physics-informed neural networks for forward and inverse PDE problems. Comput. Methods Appl. Mech. Eng. 393, 114823. http://dx.doi.org/10.1016/j.cma.2022.114823.

Zhang, Z., Li, J., Liu, B., 2024. Annealed adaptive importance sampling method in PINNs for solving high dimensional partial differential equations. J. Comput. Phys. http://dx.doi.org/10.1016/j.jcp.2024.113561, 113561–113561.